

Merkle-Chained Evidence Ledger - MCEL 1.0

Revision 1.0a, January 31 2025

John G. Underhill - contact@qrcscorp.ca

The Merkle Chain Evidence Ledger (MCEL) is a cryptographically verifiable, append-only ledger system designed to provide strong integrity, ordering, and non-repudiation guarantees for structured records. MCEL uses deterministic cSHAKE-256 domain-separated commitments, payload commitments, Merkle tree block aggregation, signed checkpoint commitment chaining, and anchored external references to ensure that recorded data cannot be altered, removed, reordered, or substituted without detection.

	Page
<u>Foreword</u>	3
1. <u>Introduction</u>	4
2. <u>Scope</u>	5
3. <u>Terms and Definitions</u>	7
4. <u>Cryptographic Primitives</u>	12
5. <u>System Architecture</u>	14
6. <u>Protocol and Data Structures</u>	17
7. <u>Operational Flow</u>	24
8. <u>Mathematical Description</u>	37
9. <u>MCEL API</u>	41
10. <u>Storage and Persistence Model</u>	49
11. <u>Security Analysis</u>	53
12. <u>Cryptanalysis Considerations</u>	60
13. <u>Deployment Considerations</u>	65
14. <u>Compliance and Interoperability</u>	69
15. <u>Conclusion</u>	73

Foreword

This document is intended as the preliminary technical specification of the Merkle Chain Evidence Ledger (MCEL), and as a reference from which conforming implementations may be developed. It is written to describe the design, structure, and operational behavior of the MCEL system in a complete and implementation-oriented manner, while remaining independent of any specific deployment or application domain.

This specification represents the initial revision of MCEL. As the system evolves through implementation, testing, and external review, revisions to this document may be required. Revision numbers shall be incremented to reflect substantive changes to the protocol, data structures, or security model. Readers are advised to consult only the most recent revision of this document as the authoritative description of MCEL.

The MCEL design emphasizes deterministic behavior, cryptographic verifiability, and long-term integrity. The system is intended to support evidentiary, audit, and compliance use cases in environments where records must remain tamper-evident and independently verifiable over extended time horizons, including in the presence of partial system compromise or untrusted storage.

The author of this specification is John G. Underhill, and may be contacted at contact@qrcscorp.ca. The MCEL design and associated reference implementations are the intellectual property of John G. Underhill and Quantum Resistant Cryptographic Solutions Corporation. The code described herein is copyrighted, and all rights are reserved.

1. Introduction

The Merkle Chain Evidence Ledger (MCEL) is a cryptographically verifiable, append-only ledger system designed to provide strong guarantees of data integrity, ordering, and provenance. MCEL is intended for use in environments where records must be preserved in a form that is tamper-evident, independently auditable, and resistant to both accidental corruption and adversarial modification.

At its core, MCEL commits payloads and record headers individually, aggregates ordered record commitments into sealed Merkle blocks, and chains signed checkpoint commitments through a previous-checkpoint field. A block Merkle root reflects the ordered records inside a block. A checkpoint commitment reflects the checkpoint header, block Merkle root, and prior checkpoint commitment, thereby representing ledger state progression over time.

MCEL is not a consensus system and does not rely on distributed agreement or economic incentives. Instead, it is designed as a single-writer or controlled-writer ledger that emphasizes verifiability, reproducibility, and long-term evidentiary value. The system assumes that integrity verification may be performed by external parties who do not trust the storage medium, the hosting environment, or the operator of the ledger itself.

To support these goals, MCEL introduces explicit concepts of anchoring and domain separation. Anchors bind ledger state to externally verifiable reference points, enabling independent confirmation that a given ledger state existed at or before a specific moment. Domains provide a mechanism for isolating namespaces, policies, or operational contexts within a shared ledger framework, allowing multiple logical ledgers to coexist without compromising integrity or auditability.

The design of MCEL is deliberately conservative in its cryptographic choices and operational assumptions. It relies on well-understood hash-based constructions, deterministic serialization, and explicit state transitions. This approach minimizes hidden complexity and reduces the risk of subtle implementation errors, while facilitating formal reasoning, code review, and third-party assessment.

This document describes the MCEL system at the protocol and data-structure level. It defines the cryptographic primitives, record formats, operational flows, and verification procedures required to construct and validate an MCEL ledger. Implementation-specific details are discussed only insofar as they affect interoperability, correctness, or security, with the intent that independent implementations following this specification will produce equivalent and verifiable results.

Scope

This document specifies the Merkle Chain Evidence Ledger (MCEL) system, defining its data structures, cryptographic constructions, and operational behavior. The scope of this specification is limited to the mechanisms required to create, extend, anchor, and verify an append-only, cryptographically verifiable ledger of records.

MCEL is concerned exclusively with integrity, ordering, and verifiability. It defines how records are committed to a ledger, how those commitments evolve over time, and how third parties may independently verify the correctness and completeness of a ledger state. The specification does not prescribe application-level semantics for records, nor does it impose any policy regarding the meaning, interpretation, or legal standing of stored data.

This specification includes detailed descriptions of the following elements:

- Ledger record structure and serialization rules
- Merkle tree construction and root evolution
- Anchor creation and verification mechanisms
- Domain separation and namespace handling
- Checkpointing, sealing, and audit traversal
- Deterministic verification procedures

MCEL does not define a network protocol, consensus mechanism, or distributed replication model. It assumes that record creation and ledger extension are performed by a trusted or controlled writer, while verification may be performed by untrusted or external parties. Any transport, synchronization, or replication of ledger data is considered out of scope and may be implemented by higher-level systems.

Similarly, MCEL does not define authentication, access control, or key management policies beyond those strictly required for cryptographic verification of anchors or signatures, where applicable. Such concerns are expected to be handled by surrounding systems or operational controls.

2.1 Intended Applications

MCEL is designed to support use cases where long-term integrity and auditability are primary requirements. These include, but are not limited to, evidence preservation, compliance logging, software supply chain provenance, configuration state tracking, and regulatory or institutional audit systems. The system is suitable for environments in which ledger data may be stored on untrusted media or transferred through untrusted channels, provided that the data can later be verified against known cryptographic commitments.

2.2 Compliance Language

The key words “MUST”, “SHALL”, “SHOULD”, and “MAY” in this document are to be interpreted as described in RFC 2119. Requirements labeled as mandatory are necessary for interoperability and security. Deviations from these requirements may result in non-conforming implementations whose outputs cannot be reliably verified by other MCEL-compliant systems.

2.3 Document Organization

The remainder of this document is organized to progress from foundational concepts to detailed specification. Cryptographic primitives and terminology are defined first, followed by architectural description, data structures, operational flows, and verification procedures. Security analysis and implementation considerations are presented in later sections to assist implementers and reviewers in assessing the correctness and robustness of MCEL-based systems.

3. Terms and Definitions

This section defines the terminology used throughout the MCEL specification. The terms defined here are intended to have precise and consistent meanings within the context of this document and MCEL-conforming implementations.

3.1 Cryptographic Primitives

Hash Function

A deterministic cryptographic function that maps an input of arbitrary length to a fixed-length output, providing preimage resistance, second-preimage resistance, and collision resistance. Hash functions are the foundational primitive used throughout MCEL to construct commitments and integrity proofs.

Hash Commitment

The output of a hash function applied to structured data, used as a compact and tamper-evident representation of that data. In MCEL, commitments are used to bind records, Merkle nodes, anchors, and checkpoints.

Merkle Tree

A tree-based hash structure in which each non-leaf node is the hash of its child nodes. The root of the tree provides a single commitment to the entire set of leaf values.

Merkle Root

The final hash value produced by a Merkle tree construction. The Merkle root uniquely represents the ordered set of records committed to the tree.

3.2 Ledger Terminology

Ledger

An append-only data structure consisting of an ordered sequence of records, together with cryptographic commitments that enable verification of integrity and order.

Record

A discrete unit of data appended to the ledger. Each record contributes to the evolving cryptographic state of the ledger and cannot be altered or removed without detection.

Append-Only

A property of a data structure in which existing entries cannot be modified or deleted, and new entries may only be added at the end.

Checkpoint

A durable representation of ledger state at a specific point in time, typically consisting of a Merkle root and associated metadata. Checkpoints enable efficient verification and recovery.

3.3 Merkle Structures

Merkle Node

An internal or leaf node within a Merkle tree. Leaf nodes typically represent hashed records, while internal nodes represent hashes of child nodes.

Leaf Node

A Merkle node corresponding directly to a record commitment.

Internal Node

A Merkle node whose value is derived from the hash of two or more child nodes.

Merkle Path

An ordered set of sibling hashes that allows a verifier to recompute the Merkle root from a given leaf node.

3.4 Anchoring

Anchor

A cryptographic structure that binds a specific ledger state, typically a Merkle root, to an external reference or context. Anchors provide evidence that a ledger state existed at or before a given event or time.

Anchor Commitment

The hash or signed structure that represents an anchored ledger state.

Anchor Verification

The process of validating that an anchor correctly binds a given ledger state and has not been forged or altered.

3.5 Domains

Domain

A logical namespace within MCEL used to isolate records, policies, or operational contexts. Domains allow multiple independent ledger spaces to coexist without collision or ambiguity.

Domain Identifier

A deterministic identifier used to distinguish one domain from another within the MCEL system.

Domain Separation

The practice of incorporating domain-specific data into cryptographic commitments to prevent cross-domain collisions or substitution attacks.

3.6 Serialization and Encoding

Canonical Serialization

A deterministic encoding of structured data such that the same logical input always produces identical byte output. Canonical serialization is mandatory for all data contributing to cryptographic commitments.

Byte Array

An ordered sequence of octets used as the basic unit of serialized data.

3.7 Verification and Audit

Verification

The process of independently confirming the correctness and integrity of a ledger, record, anchor, or checkpoint using cryptographic evidence.

Audit Traversal

The process of walking a ledger's records and associated Merkle structures to confirm completeness, order, and integrity.

3.8 Normative References

This section lists the cryptographic primitives referenced by this specification. It includes the primitives required by MCEL itself, as well as primitives used by common MCEL deployments for anchoring, attestation, or secure transport.

Hashing and XOF primitives (core to MCEL)

FIPS 202: SHA-3 Standard, Permutation-Based Hash and Extendable-Output Functions

Defines SHA3-224/256/384/512 and the SHAKE128/256 extendable-output functions. These primitives underpin MCEL commitments, Merkle node hashing, and ledger root derivation. National Institute of Standards and Technology.

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>

NIST SP 800-185: SHA-3 Derived Functions

Defines cSHAKE and KMAC among other SHA-3 derived functions. These primitives are referenced for keyed hashing, domain separation, and MAC constructions used in related QRCS systems and may be used by MCEL deployments that require keyed commitments or authenticated artifacts.

National Institute of Standards and Technology.

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf>

Authenticated encryption and conventional baselines (integration and comparison)

FIPS 197: Advanced Encryption Standard (AES)

Defines the AES block cipher, referenced as a conventional symmetric primitive baseline for systems that protect MCEL artifacts at rest or in transit.

National Institute of Standards and Technology.

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf>

NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation, Galois/Counter Mode (GCM) and GMAC

Defines AES-GCM, referenced as a conventional AEAD baseline for protecting MCEL artifacts over untrusted networks and for interoperability in systems that already standardize on AES-GCM.

National Institute of Standards and Technology.

<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>

Post-quantum public-key primitives (anchoring, attestation, secure channel integration)

FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)

Defines ML-DSA (Dilithium). Referenced for signing anchors, checkpoints, ledger attestations, and any external authenticity layer applied to MCEL state.

FIPS 204, Module-Lattice-Based Digital Signature Standard.

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>

SPHINCS+ (Hash-Based Digital Signatures)

Defines the SPHINCS+ stateless hash-based signature scheme. Referenced as an alternative signature primitive for anchors, checkpoint signing, and long-term attestations where hash-based security assumptions are preferred.

FIPS 205, Stateless Hash-Based Digital Signature Standard.

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf>

FIPS 203: Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM)

Defines ML-KEM (Kyber). Referenced for establishing post-quantum secure channels that transport MCEL artifacts, distribute checkpoints, or support replicated storage deployments.

FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard.

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>

Classic McEliece (Code-Based Key Encapsulation)

Defines the Classic McEliece post-quantum KEM candidate (code-based). Referenced for deployments preferring conservative code-based hardness for secure channel establishment related to MCEL distribution or federation.

NIST Post-Quantum Cryptography Project Classic McEliece.

<https://classic.mceliece.org/mceliece-spec-20221023.pdf>

QRCS constructions referenced by MCEL deployments

Rijndael Cryptographic Stream (RCS)

A QRCS symmetric authenticated stream cipher construction combining a Rijndael-derived wide-block design with Keccak-family primitives and KMAC-based authentication. Referenced

for deployments that standardize on QRCS symmetric primitives for protecting MCEL artifacts or for integrated systems that couple MCEL with QRCS secure transport protocols.
QRCS The RCS Specification. https://www.qrcscorp.ca/documents/rsc_specification.pdf

4. Cryptographic Primitives

MCEL relies on cSHAKE-256 domain-separated hash commitments to provide integrity, ordering, and verifiability guarantees. The mandatory commitment output length is 32 bytes. The cSHAKE name string is MCEL-LEDGER, and each object class uses a fixed customization string. This design avoids ambiguity between payload commitments, record commitments, Merkle internal nodes, block commitments, checkpoint commitments, and anchor commitments.

The notation $H_S(M)$ denotes cSHAKE-256 with output length 32 bytes, name string $N = \text{MCEL-LEDGER}$, customization string S , and message input M . All commitment formulas in this document are defined in terms of this function family.

Table 4.1 - MCEL cSHAKE-256 Commitment Domains

Name	Customization String	Purpose	Output Size
Block	MCEL-DOMAIN-BLCK	Block commitment	32 bytes
Checkpoint	MCEL-DOMAIN-CHCK	Checkpoint commitment	32 bytes
Ciphertext	MCEL-DOMAIN-CTXT	Encrypted payload commitment	32 bytes
Node	MCEL-DOMAIN-NODE	Merkle internal node and empty root	32 bytes
Plaintext	MCEL-DOMAIN-PTXT	Plaintext payload commitment	32 bytes
Record	MCEL-DOMAIN-RCRD	Record commitment	32 bytes
Anchor	MCEL-DOMAIN-ANCR	Anchor commitment	32 bytes

4.1 Hash Functions

MCEL uses cSHAKE-256 as its mandatory commitment primitive. The hash function output length SHALL be 32 bytes for all MCEL commitments. The common cSHAKE name string SHALL be MCEL-LEDGER, and the customization string SHALL be the commitment domain associated with the artifact being computed.

All hash operations within MCEL are deterministic. Given identical input byte sequences, the hash output MUST be identical across all conforming implementations. Any deviation in serialization, encoding, or hashing behavior results in incompatible ledger states and invalid verification results.

The hash function output length defines the security level of the ledger. Implementations SHOULD select a hash function with a security margin appropriate for long-term integrity and evidentiary use.

4.2 Record Commitments

Each ledger record is committed through a two-stage construction. The payload is first committed under either the plaintext or ciphertext payload domain. The record commitment is then computed as $H_RCRD(Enc(record_header) || payload_commitment)$. The raw payload is stored with the record but is not directly concatenated into the record commitment input.

Record commitments serve as the leaf values of the Merkle structure. Once a record commitment has been incorporated into the ledger, it becomes cryptographically bound to all subsequent ledger state. Any modification to a record or its position in the ledger necessarily alters the resulting Merkle root.

4.3 Merkle Tree Construction

MCEL employs a deterministic binary Merkle tree to aggregate the ordered record commitments of a sealed block. Leaf values are record commitments. Internal nodes are computed as $H_NODE(left_hash || right_hash)$, where both child values are 32 bytes. If a level has an unpaired final node, that node is paired with itself and hashed as $H_NODE(last_hash || last_hash)$.

The Merkle tree construction used by MCEL is strictly ordered. The position of a record within the ledger directly affects the structure of the tree and the resulting root. This property ensures that record ordering is cryptographically enforced and verifiable.

All internal node hashes MUST be computed using canonical concatenation of child node values. Implementations SHALL NOT introduce implicit padding, length ambiguity, or non-deterministic ordering when constructing Merkle nodes.

4.4 Merkle Roots and Ledger State

The block Merkle root represents a compact commitment to the ordered set of record commitments inside a sealed block. Ledger state is identified by the checkpoint commitment chain, not by the block Merkle root alone. A checkpoint commitment binds the checkpoint header, block Merkle root, and previous checkpoint commitment.

Successive durable ledger states produce a sequence of checkpoint commitments, each reflecting a sealed block and the previous checkpoint commitment. Earlier checkpoint commitments remain valid references to prior states and MAY be retained for historical verification or checkpointing purposes.

4.5 Anchors and External Commitments

Anchors bind a checkpoint commitment to an external context through an encoded anchor reference. The anchor commitment is $H_ANCR(\text{checkpoint_commitment} || \text{Enc}(\text{anchor_reference}))$. Anchors provide integrity binding to an external reference; signer authenticity or time authority must be supplied by the external anchoring profile if required.

The cryptographic strength of an anchor depends on both the underlying hash function and the integrity of any external data incorporated into the anchor commitment. MCEL treats anchors as verifiable evidence structures rather than trusted assertions.

4.6 Domain Separation

Cryptographic domain separation is achieved through cSHAKE-256 customization strings, not through a variable `domain_id` field in the record header. The namespace identifier is an operational ledger identifier used for storage and state association, while the MCEL cSHAKE domains separate commitment types cryptographically.

Domain identifiers **MUST** be included in all cryptographic commitments that depend on domain context, including record commitments, Merkle roots, and anchors. This prevents cross-domain replay and substitution attacks and ensures clear namespace isolation.

4.7 Security Assumptions

The security of MCEL reduces to the assumption that the underlying hash function remains collision resistant and that canonical serialization rules are followed exactly. No additional hardness assumptions are introduced by the system.

If the hash function is broken with respect to collision resistance or second-preimage resistance, the integrity guarantees of MCEL are weakened accordingly. For this reason, MCEL is designed to permit hash function replacement in future revisions, provided that deterministic behavior and serialization rules are preserved.

5. System Architecture

This section describes the architectural components of the Merkle Chain Evidence Ledger (MCEL) and the relationships between them. MCEL is structured as a layered system in which cryptographic commitments, state progression, and verification are cleanly separated. This architecture is intended to support deterministic behavior, independent verification, and long-term auditability.

5.1 Architectural Overview

MCEL is organized as a layered local ledger. Records are appended to a record stream, committed into sealed Merkle blocks, and then bound into a signed checkpoint commitment chain. The head checkpoint is the authoritative ledger-state reference loaded during initialization and used for recovery.

The system is composed of the following primary architectural components:

- Policy, key rotation, query, and auxiliary index support layers
- Anchor reference and anchor commitment layer
- Checkpoint commitment chain and signed checkpoint bundle layer
- Block Merkle construction and block sealing layer
- Record and payload commitment layer
- Ledger core and storage callback interface
 - The ledger core
 - The Merkle construction layer
 - The anchoring mechanism
 - The domain separation mechanism

These components operate together to ensure that ledger state is immutable, verifiable, and unambiguously scoped.

5.2 Ledger Core

The ledger core is responsible for managing the ordered sequence of records and enforcing append-only semantics. It defines the lifecycle of a record from creation through commitment and inclusion in the ledger state.

The ledger core ensures that:

- Records are appended in a well-defined order

- Existing records are never modified or removed
- Each append operation results in a new, well-defined ledger state

The ledger core does not interpret record contents. Records are treated as opaque byte sequences whose meaning is defined by higher-level systems.

5.3 Merkle Construction Layer

The Merkle construction layer aggregates record commitments into a hierarchical hash structure. Each record commitment forms a leaf node in the Merkle tree. Internal nodes are computed deterministically from their child nodes, and the root node represents a compact commitment to the complete ordered record set.

The Merkle construction layer provides:

- Efficient integrity verification of individual records
- Compact representation of complete ledger state
- Cryptographic enforcement of record ordering

Merkle roots produced by this layer are the primary integrity reference used by all verification, anchoring, and audit operations.

5.4 Anchoring Mechanism

The anchoring mechanism binds a specific ledger state to an external reference or context. Anchors are constructed from Merkle roots together with additional contextual data defined by the anchoring profile.

Anchors enable third parties to verify that a particular ledger state existed at or before a given external event or reference point. The anchoring mechanism does not require trust in the ledger operator, storage medium, or transport channel.

Anchoring is optional but normative. When anchors are used, their construction and verification MUST follow the rules defined by this specification to ensure interoperability and evidentiary value.

5.5 Domain Separation

MCEL supports domain separation to allow multiple independent logical ledgers to coexist within a shared framework. A domain defines a namespace and context that is incorporated into cryptographic commitments.

Domain separation ensures that:

- Identical record data in different domains cannot collide

- Records and anchors cannot be substituted across domains
- Verification is unambiguous with respect to context

Domain identifiers are incorporated into record commitments, Merkle construction, and anchoring operations as required.

5.6 State Progression and Checkpoints

Ledger state progresses monotonically when checkpoints are sealed. Record append creates stored records and record commitments, block sealing creates block roots and block commitments, and checkpoint sealing advances the head checkpoint commitment by binding the current block root to the previous checkpoint commitment.

Checkpoints are representations of ledger state and do not alter the cryptographic construction. They serve as optimization and durability mechanisms and do not weaken the append-only or integrity properties of the ledger.

5.7 Trust Model and Assumptions

MCEL assumes a controlled or trusted writer for record creation, but does not assume trusted storage, transport, or verification environments. All integrity guarantees are enforced cryptographically and may be validated by any party with access to the ledger data and associated commitments.

No assumptions are made regarding network connectivity, synchronization, or consensus. These concerns are intentionally excluded from the MCEL architecture and may be addressed by external systems.

6. Protocol and Data Structures

This section defines the concrete data structures, packets, and data types used by the Merkle Chain Evidence Ledger (MCEL). The structures defined here are normative and describe the in-memory and serialized representations used to construct, extend, anchor, and verify an MCEL ledger. The intent is to make the protocol mechanically precise while remaining implementation-agnostic.

MCEL does not define a network packet protocol. Instead, it defines structured data objects that MAY be stored, transmitted, or embedded within higher-level systems. For consistency with established QRCS specifications, these objects are described using tables that enumerate fields, types, sizes, and functions.

6.1 MCEL Record Structure

An MCEL record represents a single append-only entry in the ledger. Records are opaque with respect to application semantics, but their structure and serialization are strictly defined to ensure deterministic commitments.

An MCEL record consists of a fixed-size encoded header, a 32-byte payload commitment, and a payload. The header binds ordering metadata, type, flags, version, payload length, and key identifier. Cryptographic object separation is provided by cSHAKE customization strings rather than by a variable domain_id field in the record header.

Table 6.1a - MCEL Record Structure

Data Name	Data Type	Size	Function
sequence	uint64	8 bytes	Record sequence number within the ledger
timestamp	uint64	8 bytes	Advisory UTC or monotonic timestamp
payload_len	uint32	4 bytes	Length of the payload bytes
type	uint32	4 bytes	Record type identifier
flags	uint8	1 byte	Record flags; bit 0 indicates encrypted payload
version	uint8	1 byte	Record format version
keyid	uint8 array	32 bytes	Key, signer, or policy identifier

The record header ensures that records are self-describing with respect to sequence, timestamp, payload length, type, flags, version, and key identifier. The version field prevents cross-version substitution. The encrypted flag selects the plaintext or ciphertext payload-commitment domain. The payload length is included explicitly to ensure canonical serialization.

6.2 Record Commitment

A record commitment is the cSHAKE-256 record-domain commitment over the canonical encoded record header and the 32-byte payload commitment. The payload commitment is computed separately under the plaintext or ciphertext payload domain.

The commitment is not stored as part of the record itself but is derived during append operations and incorporated into the Merkle structure.

Table 6.1b - Record Commitment

Data Name	Data Type	Size	Function
payload_commitment	uint8 array	32 bytes	H_PTXT(payload) or H_CTXT(payload)
record_hash	uint8 array	32 bytes	H_RCRD(encoded_header payload_commitment)

The record commitment binds together the encoded record header and payload commitment. Record ordering is bound when ordered record commitments are incorporated into the block Merkle root.

6.3 Merkle Node Structure

Merkle nodes are computed values used to aggregate record commitments into a block Merkle root. MCEL does not require serialized Merkle node objects for normal operation.

Table 6.2a - Merkle Node Structure

Data Name	Data Type	Size	Function
leaf_value	uint8 array	32 bytes	Record commitment used as Merkle leaf
internal_node	uint8 array	32 bytes	H_NODE(left_hash right_hash)
odd_node_rule	procedure	deterministic	Unpaired final node is hashed with itself
empty_root	uint8 array	32 bytes	H_NODE(empty string)

Leaf nodes are record commitments. Internal nodes are computed as H_NODE(left_hash || right_hash). If a level contains an unpaired final node, that node is paired with itself and hashed as H_NODE(last_hash || last_hash).

6.4 Merkle Root

The block Merkle root represents the ordered record commitments inside a sealed block. The durable ledger state is identified by the checkpoint commitment and checkpoint header.

Table 6.2b - Merkle Root Structure

Data Name	Data Type	Size	Function
-----------	-----------	------	----------

block_merkle_root	uint8 array	32 bytes	Commitment to ordered records inside a sealed block
record_count	uint32	4 bytes	Number of records committed by the block root
checkpoint_commitment	uint8 array	32 bytes	Ledger-state commitment binding block root and previous checkpoint

The record count binds a block root to the number of records covered by that block. The checkpoint commitment binds that block root into the checkpoint chain and prevents truncation or extension ambiguity when compared to a retained checkpoint reference.

6.5 Anchor Structure

An anchor binds a checkpoint commitment to an external reference encoded as an anchor reference. The anchor commitment is $H_ANCR(\text{checkpoint_commitment} || \text{encoded_anchor_reference})$.

Table 6.3a - Anchor Structure

Data Name	Data Type	Size	Function
version	uint8	1 byte	Anchor reference format version
flags	uint8	1 byte	Anchor reference flags
type	uint8	1 byte	Anchor reference type
reserved	uint8	1 byte	Reserved; SHALL be zero
chain_id_len	uint16	2 bytes	Length of chain identifier
reference_len	uint16	2 bytes	Length of external reference
chain_id	uint8 array	Variable	External chain or registry identifier
reference	uint8 array	Variable	External transaction or witness bytes

The encoded anchor reference MAY contain external chain identifiers, witness identifiers, transaction identifiers, timestamps, or document references. The anchor commitment provides integrity binding. Signer authentication or time authority is supplied only by an external anchoring profile.

6.6 Domain Structure

MCEL distinguishes operational namespaces from cryptographic commitment domains. The namespace identifier associates a ledger instance with storage and policy context. Cryptographic domain separation is provided by fixed cSHAKE customization strings.

Table 6.4a - Domain Structure

Data Name	Data Type	Size	Function
namespace_id	uint8 array	0 to 64 bytes	Operational ledger namespace
cSHAKE name	string	11 bytes	MCEL-LEDGER
customization	string	16 bytes	Commitment-domain string
version	uint8	1 byte	Committed artifact version

The cSHAKE name string and customization string are incorporated into all MCEL commitment computations. A variable domain_id field is not part of the encoded record header.

6.7 Checkpoint Structure

A checkpoint captures a durable snapshot of ledger state for recovery, audit, or long-term storage.

Table 6.5a - Checkpoint Structure

Data Name	Data Type	Size	Function
chk_sequence	uint64	8 bytes	Checkpoint sequence number
first_record_seq	uint64	8 bytes	First record sequence in covered block
timestamp	uint64	8 bytes	Advisory checkpoint timestamp
record_count	uint32	4 bytes	Number of records in covered block
flags	uint8	1 byte	Checkpoint flags
version	uint8	1 byte	Checkpoint format version
keyid	uint8 array	32 bytes	Checkpoint signer or policy identifier

Checkpoints do not alter ledger semantics. They are derived artifacts used to optimize verification and recovery.

6.8 Serialization and Canonical Encoding

All MCEL structures defined in this section **MUST** be serialized using canonical encoding rules. Fields **MUST** be encoded in the order listed in the tables. Integer fields **MUST** use fixed-width encodings with defined endianness. Variable-length fields **MUST** be length-prefixed or otherwise unambiguously delimited.

Canonical serialization is a security-critical requirement. Any deviation results in incompatible commitments and invalid verification outcomes.

The encoded record header size is 58 bytes. The encoded block header size is 62 bytes. The encoded checkpoint header size is 62 bytes. All uint16, uint32, and uint64 values are big-endian. Variable-length anchor fields are length-prefixed with uint16 big-endian lengths.

Table 6.6a - Encoded MCEL Block Header

Data Name	Data Type	Size	Function
block_sequence	uint64	8 bytes	Block sequence number
first_record_seq	uint64	8 bytes	First record sequence covered
timestamp	uint64	8 bytes	Advisory block timestamp
record_count	uint32	4 bytes	Number of records in block
flags	uint8	1 byte	Block flags
version	uint8	1 byte	Block format version
keyid	uint8 array	32 bytes	Block signer or policy identifier

Table 6.6b - Sealed Block Layout

Data Name	Data Type	Size	Function
encoded_block_header	uint8 array	62 bytes	Canonical block header
block_merkle_root	uint8 array	32 bytes	Merkle root over record commitments
block_commitment	uint8 array	32 bytes	H_BLK(encoded block header block Merkle root)
record_commitments	uint8 array	32 * record_count	Ordered record commitments

Table 6.7a - Checkpoint Bundle Layout

Data Name	Data Type	Size	Function
encoded_checkpoint_header	uint8 array	62 bytes	Canonical checkpoint header
block_merkle_root	uint8 array	32 bytes	Covered block Merkle root
previous_checkpoint_commitment	uint8 array	32 bytes	Previous checkpoint commitment or zero for genesis
signed_checkpoint_commitment	uint8 array	signature size + 32	Signature object over checkpoint commitment

Table 6.8a - Key Rotation Payload

Data Name	Data Type	Size	Function
version	uint8	1 byte	Key rotation payload version
reserved	uint8	1 byte	Reserved
old_keyid	uint8 array	32 bytes	Identifier of previous key
public_key_len	uint16	2 bytes	Length of new verification key
public_key	uint8 array	public_key_len	New verification key bytes

Table 6.9a - Logical Storage Locations

Name	Location	Semantics	Contents
Records	mccl/records	Append-only	Stored records
Blocks	mccl/blocks	Append-only	Sealed blocks
Checkpoints	mccl/checkpoints	Append-only	Checkpoint bundle log
Head	mccl/head	Overwrite or latest object	Current head checkpoint bundle

7. Operational Flow

This section specifies the operational behavior of the Merkle Chain Evidence Ledger (MCEL). It defines the end-to-end lifecycle of a ledger instance, including initialization, record append, Merkle commitment updates, checkpoint creation, sealing operations, anchoring, and verification. MCEL operation is a deterministic sequence of state transitions. The resulting commitments, including record commitments, Merkle roots, checkpoints, and seals, **MUST** be reproducible across conforming implementations given identical inputs.

MCEL does not define a transport protocol. The flows described in this section are local or system-level operations that **MAY** be invoked by an application, service, or higher-level protocol.

7.1 Ledger Initialization

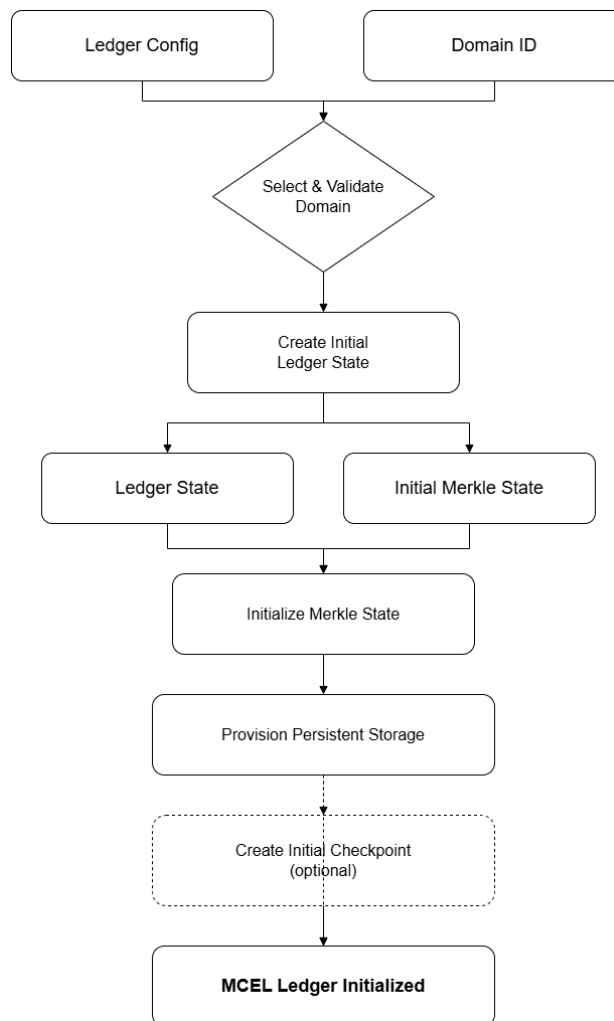


Figure 7.1: MCEL ledger initialization and domain binding.

A conforming implementation SHALL provide a ledger initialization procedure that creates a new ledger instance bound to a specific domain. The initialization procedure establishes the cryptographic context and persistent storage context for all subsequent operations.

Initialization MUST perform the following operations:

1. **Domain selection and validation**

The implementation SHALL select a domain identifier and validate that it is well-formed according to Section 6.6. If the domain already exists in persistent storage, initialization SHALL either (a) open the existing domain state, or (b) reject initialization, depending on the selected startup mode.

2. **Ledger state creation**

The implementation SHALL instantiate an empty ledger state, including any required counters, sequence values, and state identifiers. The empty ledger state MUST be uniquely associated with the domain identifier.

3. **Merkle state initialization**

The Merkle construction layer SHALL be initialized to an empty state. The initial Merkle root for an empty ledger SHALL be defined deterministically by the implementation profile. If the profile defines an explicit empty-root constant, that value SHALL be used. If the profile defines an empty-root derivation procedure, that procedure SHALL be used.

4. **Storage provisioning (if applicable)**

If persistent storage is enabled, the implementation SHALL provision required storage artifacts, such as ledger directories, index files, record containers, or metadata stores. Storage operations MUST be performed in a manner that preserves atomicity for subsequent append operations.

5. **Optional initial checkpoint**

Implementations MAY create an initial checkpoint representing the empty state, particularly in environments requiring deterministic bootstrapping and recoverability.

The ledger is considered initialized when the domain is bound, the Merkle state is defined, and the persistent context is ready to accept append operations.

7.2 Record Creation and Preparation

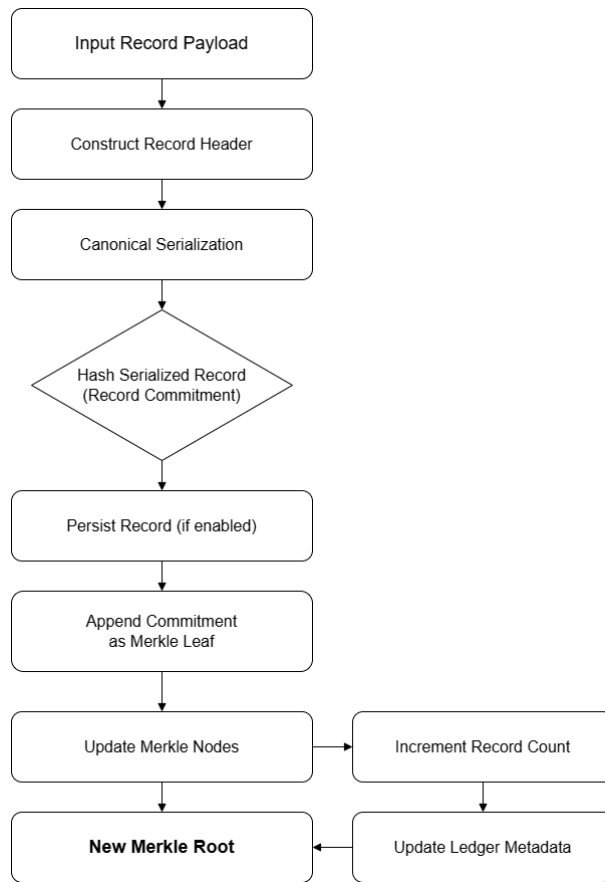


Figure 7.2: MCEL record construction and commitment derivation.

MCEL treats records as opaque payloads with a structured header. Record creation is an application-facing step that prepares data for deterministic commitment and append.

A record prepared for append **MUST** satisfy the following:

1. **Header construction**

The record header **SHALL** be populated with the required fields defined in Section 6.1, including version, domain identifier, flags, and payload length. All numeric fields **MUST** use the defined canonical encodings.

2. **Payload construction**

The payload **SHALL** be populated with application-defined bytes. The payload length field **MUST** match the exact byte length of the payload.

3. **Canonical serialization**

The record **MUST** be serialized according to the canonical encoding rules in Section 6.8. Serialization **MUST** be stable and unambiguous. Any deviation constitutes a protocol violation.

4. Record commitment derivation

The record commitment SHALL be computed by hashing the canonical serialized record.

The record commitment is the leaf input to the Merkle construction.

Record preparation is complete when the canonical serialized record and its commitment are available to the append procedure.

7.3 Append Operation

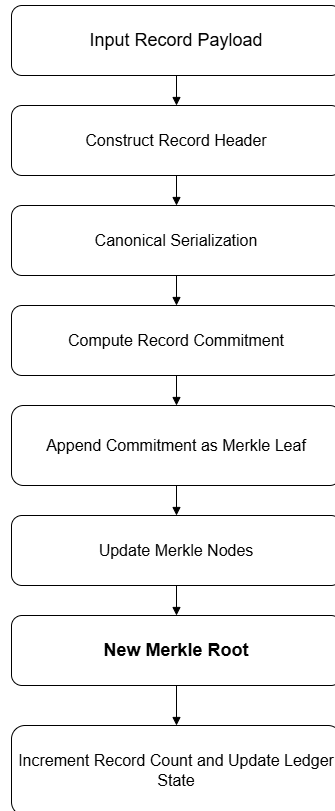


Figure 7.3: MCEL record append and Merkle update.

The append operation stores a prepared record and returns its record commitment. It does not by itself advance the checkpoint-chain state. Ledger state advances when a block is sealed and then a checkpoint is sealed. This separation allows records to be batched into Merkle blocks before checkpoint commitment.

A conforming implementation SHALL perform the append operation as follows:

1. Pre-append validation

The implementation SHALL validate that the ledger is initialized, that the record domain identifier matches the ledger domain, and that the record fields are well-formed. If validation fails, the append operation SHALL fail without modifying ledger state.

2. Record persistence (if applicable)

If persistent storage is enabled, the implementation SHALL write the canonical serialized record to stable storage. The write MUST be performed in a way that supports recovery. Implementations SHOULD avoid committing Merkle state changes until the record write is durable.

3. Leaf incorporation

The record commitment SHALL be incorporated into the Merkle construction as the next leaf in the ordered sequence. The leaf position is determined by the current record count, which SHALL be incremented exactly once for each successful append.

4. Merkle node updates

The Merkle construction layer SHALL update internal nodes deterministically, producing a new Merkle root. The update MUST be equivalent to recomputing the Merkle root over the full ordered set of record commitments.

5. Ledger state update

The ledger state SHALL be updated to reflect the new record count and new Merkle root. The new Merkle root SHALL become the authoritative identifier of the updated ledger state.

6. Post-append persistence (if applicable)

If persistent storage is enabled, the updated ledger metadata, including record count and Merkle root, SHALL be committed durably. The implementation SHOULD ensure that either both the record and updated metadata are durable, or neither is durable, using atomic update strategies appropriate to the storage medium.

A record is considered appended only when the new ledger state has been established and, if applicable, persisted.

7.4 Checkpoint Creation

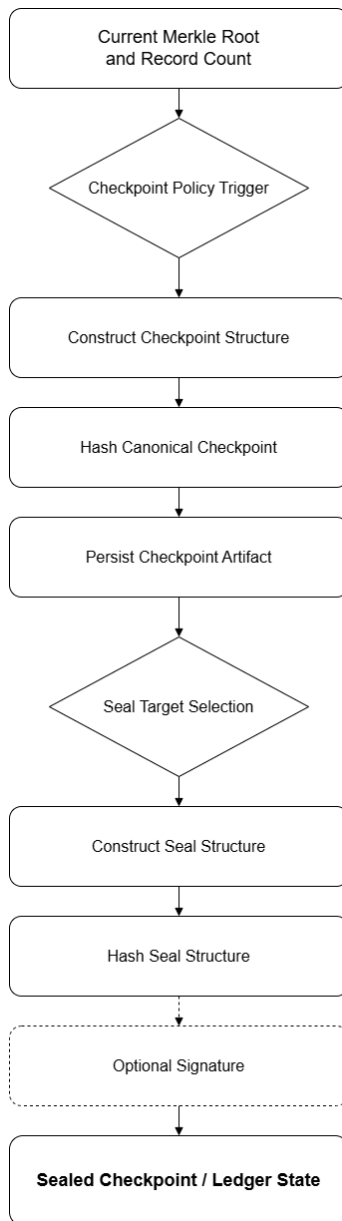


Figure 7.4: MCEL checkpoint creation and storage.

A checkpoint is a durable representation of ledger state at a specific point. Checkpoints are used to accelerate verification, support recovery, and provide stable audit references.

A conforming implementation that supports checkpoints SHALL create a checkpoint as follows:

1. **Checkpoint selection**

The implementation SHALL select a ledger state to checkpoint, identified by a Merkle root and record count. The selection MAY be periodic, policy-driven, or triggered by application events.

2. **Checkpoint structure construction**

The checkpoint structure SHALL be populated with version, domain identifier, Merkle root, and record count, as defined in Section 6.7.

3. **Checkpoint commitment derivation**

The checkpoint hash SHALL be computed over the canonical serialization of the checkpoint structure. This hash provides a compact identifier for the checkpoint artifact itself.

4. **Checkpoint persistence**

The checkpoint structure and its hash SHALL be stored durably. The implementation SHOULD ensure that a stored checkpoint is recoverable even if a failure occurs during subsequent append operations.

Checkpoints do not modify ledger semantics. They are derived artifacts that reference an existing ledger state.

7.5 Sealing Operations

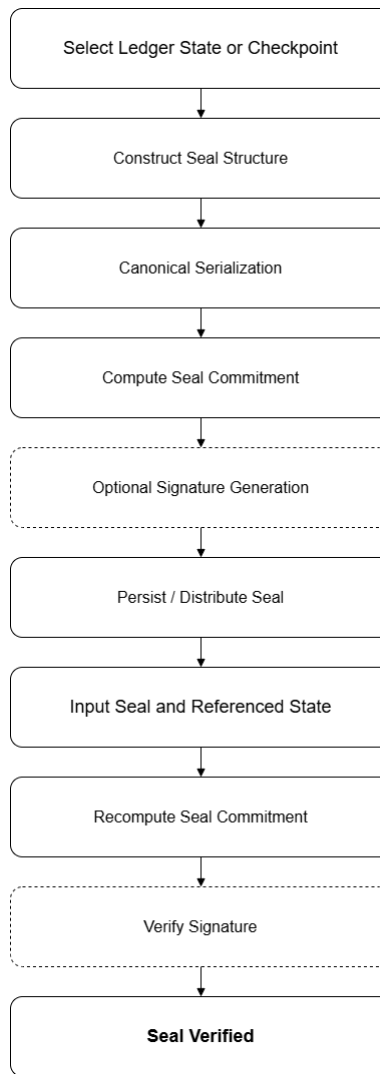


Figure 7.5: MCEL seal creation and seal verification.

Sealing is the operation of producing a cryptographic artifact that finalizes or attests to a ledger state. Seals are used to provide evidentiary stability and to prevent ambiguous interpretation of ledger progression when artifacts are exchanged between parties.

A sealing procedure SHALL perform the following steps:

1. **Seal target selection**

The implementation SHALL select a target state to seal, typically a checkpoint or current Merkle root and record count.

2. **Seal structure construction**

The seal structure SHALL include, at minimum, the domain identifier and the target Merkle root, and SHOULD include the record count and checkpoint hash when sealing a checkpointed state.

3. Seal commitment derivation

The seal commitment SHALL be computed as a hash over the canonical serialized seal structure. This commitment SHALL be used as the primary identifier of the seal.

4. Optional signature or authentication

If the implementation profile requires authentication, the seal commitment or seal structure SHALL be signed using a configured signature scheme. When used, the signature SHALL be computed over canonical bytes and SHALL be included with the seal artifact.

5. Seal persistence and publication

The seal artifact SHALL be stored durably and MAY be published to external systems.

Seal verification SHALL recompute the seal commitment from the referenced ledger state and compare it to the stored seal commitment. If signatures are present, signature verification SHALL be performed before the seal is accepted as valid.

7.6 Anchoring

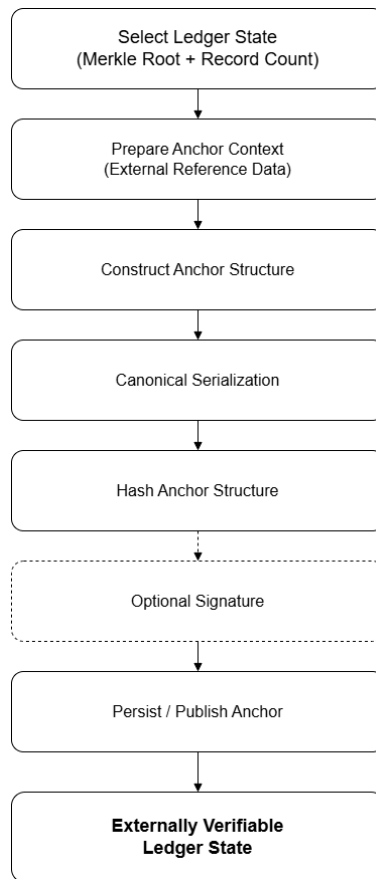


Figure 7.6: MCEL anchor creation and external binding.

Anchoring binds a ledger state to an external context, enabling third parties to verify that a given Merkle root existed at or before an externally defined reference point.

Anchor creation SHALL perform the following operations:

In MCEL, the anchor target is the checkpoint commitment, not the raw block Merkle root. The anchor reference is encoded as a canonical variable-length structure and the anchor commitment is $H_ANCR(\text{checkpoint_commitment} \parallel \text{encoded_anchor_reference})$.

1. **Anchor target selection**

The implementation SHALL select a target ledger state, typically the current Merkle root or a checkpointed state.

2. **Anchor context construction**

The anchor context SHALL be populated with external reference data. Examples include timestamps, external transaction identifiers, document references, or higher-level system commitments. The anchor context MUST be serialized deterministically.

3. **Anchor structure construction**

The anchor structure SHALL include the anchor version, domain identifier, Merkle root, anchor context, and optional signature field (Section 6.5).

4. **Anchor commitment derivation**

The anchor commitment SHALL be computed over the canonical serialization of the anchor structure. This commitment SHALL be used as the anchor identifier.

5. **Optional signature**

If the anchoring profile requires authentication, the anchor structure or anchor commitment SHALL be signed and the signature included with the anchor artifact.

Anchor verification SHALL recompute the anchor commitment and validate any signatures present. The verifier SHALL additionally confirm that the referenced Merkle root is a valid ledger state for the referenced domain.

7.7 Ledger Verification and Audit Traversal

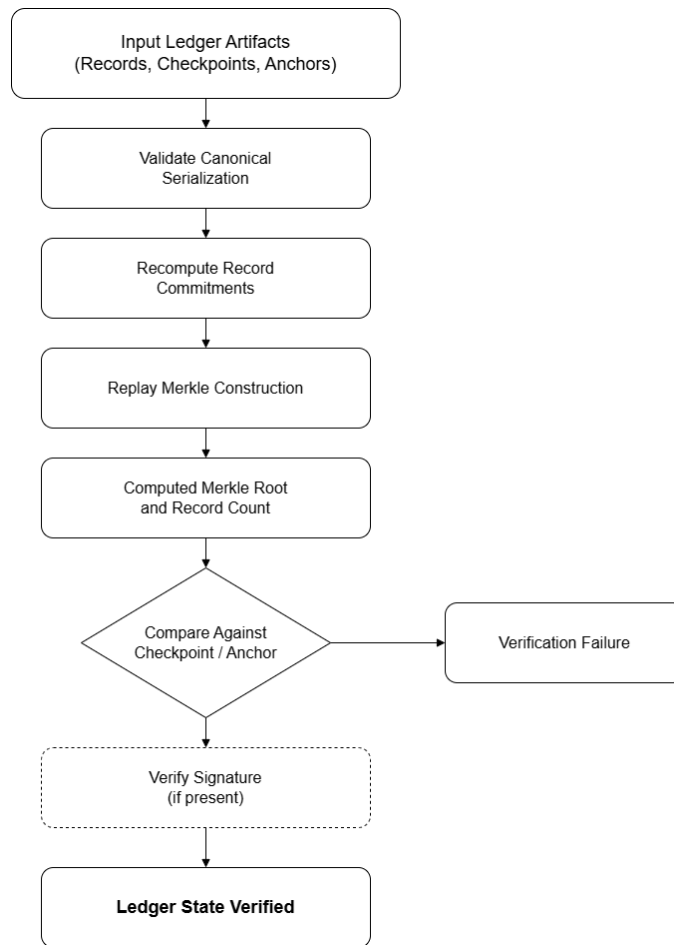


Figure 7.7: MCEL verification and audit traversal.

Verification is the process by which a verifier confirms the integrity and correctness of a ledger state and its derived artifacts.

A conforming verification procedure SHALL support the following modes:

1. **Full ledger verification**

The verifier reconstructs record commitments from canonical serialized records, rebuilds the Merkle structure deterministically, and compares the resulting Merkle root and record count to an asserted state (checkpoint, seal, or anchor).

2. **Checkpoint-based verification**

The verifier validates a checkpoint hash and then verifies append operations after the checkpoint by replaying record commitments and updating the Merkle state until the target state is reached.

3. **Inclusion verification (Merkle proof)**

The verifier confirms that a specific record commitment is included in a ledger state by

validating a Merkle path against a known Merkle root. The proof MUST bind to a specific leaf position and domain context.

Verification SHALL fail if any of the following conditions occur:

- Canonical serialization fails or is inconsistent
- Any record commitment mismatch is detected
- The reconstructed Merkle root differs from the asserted root
- Record count differs from the asserted record count
- Anchor or seal authentication fails, where authentication is required

Audit traversal is the process of iterating records and derived commitments to produce an evidentiary report. Audit output formats are out of scope, but audit procedures MUST be based on the normative verification steps defined here.

7.8 Recovery Behavior

Figure 7.8: MCEL recovery using persisted records and checkpoints.

(Placeholder for diagram illustrating restart recovery from last checkpoint, replay of subsequent records, and reestablishment of current Merkle root.)

When persistent storage is used, implementations SHALL support recovery following interruption or failure. Recovery SHALL reestablish ledger state deterministically using stored artifacts.

A recovery procedure SHALL:

1. Load the most recent valid checkpoint, if checkpoints are used
2. Replay subsequent stored records in order, recomputing commitments and Merkle updates
3. Reconstruct the current Merkle root and record count
4. Validate reconstructed state against stored metadata, seals, or anchors, if present

Recovery MUST reject any stored artifacts that cannot be validated under canonical serialization and hash verification rules.

A present head checkpoint bundle that cannot be read exactly or cannot be verified SHALL cause initialization failure. An implementation SHALL NOT silently treat a corrupt head as an empty ledger state.

8. Mathematical Description

This section provides a formal mathematical description of the Merkle Chain Evidence Ledger (MCEL). The definitions and relations in this section are normative and define the cryptographic objects, state transitions, and verification predicates used by MCEL.

8.1 Notation and Preliminaries

Let H denote a cryptographic hash function mapping arbitrary-length byte strings to fixed-length outputs:

$$H : \{0,1\}^* \rightarrow \{0,1\}^n$$

The hash function H is assumed to provide preimage resistance, second-preimage resistance, and collision resistance.

Let $\parallel$ denote byte-string concatenation.

Let $\text{Enc}(\cdot)$ denote the canonical serialization function defined in Section 6.8, which maps a structured object to a unique byte-string representation.

Let $\mathbb{N}$ denote the set of non-negative integers.

All hash computations in MCEL are performed over canonically serialized inputs. Any deviation from canonical serialization invalidates the constructions defined in this section.

Implementation-equivalent commitment equations are defined as follows:

Payload commitment: $\text{PC} = H_PTXT(\text{payload})$ for plaintext payloads, or $\text{PC} = H_CTXT(\text{ciphertext})$ for encrypted payloads.

Record commitment: $\text{RC} = H_RCRD(\text{Enc}(\text{record_header}) \parallel \text{PC})$.

Merkle internal node: $N = H_NODE(\text{left_hash} \parallel \text{right_hash})$, with duplicate-last handling for odd tree levels.

Block commitment: $\text{BC} = H_BLCK(\text{Enc}(\text{block_header}) \parallel \text{block_merkle_root})$.

Checkpoint commitment: $\text{CC} = H_CHCK(\text{Enc}(\text{checkpoint_header}) \parallel \text{block_merkle_root} \parallel \text{previous_checkpoint_commitment})$.

Anchor commitment: $\text{AC} = H_ANCR(\text{checkpoint_commitment} \parallel \text{Enc}(\text{anchor_reference}))$.

8.2 Records and Record Commitments

Let R_i denote the i -th record appended to the ledger. Each record consists of a header and an opaque payload:

$$R_i = (\text{version}_i, \text{domain}_i, \text{flags}_i, \text{payload}_i)$$

The canonical byte representation of record R_i is defined as:

$$B_i = \text{Enc}(R_i)$$

The record commitment C_i is defined as the cryptographic hash of the canonical serialization:

$$C_i = H(B_i)$$

The record commitment uniquely binds the encoded record header and the payload commitment. Cumulative ledger binding is not provided by a per-record prior-commitment field; it is provided by the block Merkle root and by checkpoint commitments that include the previous checkpoint commitment.

8.3 Ordered Record Sequence

An MCEL ledger containing k records is defined as an ordered sequence:

$$L_k = (R_0, R_1, \dots, R_{k-1})$$

with corresponding record commitments:

$$C_k = (C_0, C_1, \dots, C_{k-1})$$

The ordering of records is semantically significant. Two ledgers containing identical records in different orders are considered distinct.

8.4 Merkle Tree Construction

MCEL defines a deterministic Merkle tree over the ordered sequence of record commitments.

Each leaf node L_i is defined as:

$$L_i = C_i$$

Given two child nodes x and y , the parent node P is computed as:

$$P = H(x \parallel y)$$

If the number of nodes at a given level is odd, the handling of the unpaired node SHALL be deterministic and defined by the implementation profile. All conforming implementations MUST apply the same rule.

The Merkle root M_k is the final node obtained by recursively hashing all levels of the tree constructed over C_k .

8.5 Ledger State

The durable MCEL ledger state after a sealed checkpoint is identified by the checkpoint commitment and checkpoint header. The block Merkle root identifies the ordered record

commitments inside a block, while the checkpoint commitment identifies the chained ledger state.

$$Sk = (Mk, k)$$

where Mk is the Merkle root committing to the ordered record sequence, and k is the record count.

The pair (Mk, k) uniquely identifies a ledger state.

8.6 Append Operation as a State Transition

Appending a new record Rk creates a stored record and a record commitment. Sealing a block creates a block Merkle root and block commitment. Sealing a checkpoint creates the next durable ledger state by committing to the block root and previous checkpoint commitment.

The append operation is defined as follows.

First, compute the new record commitment:

$$Ck = H(Enc(Rk))$$

Extend the commitment sequence:

$$Ck + 1 = (Ck, Ck)$$

Recompute the Merkle root over the extended sequence:

$$Mk + 1 = Merkle(Ck + 1)$$

Update the ledger state:

$$Sk + 1 = (Mk + 1, k + 1)$$

This transition is deterministic. Given identical inputs, all conforming implementations SHALL produce identical results.

8.7 Checkpoints

A checkpoint captures a specific ledger state without modifying the ledger itself.

Given a sealed block root M , checkpoint header CH , and previous checkpoint commitment $Prev$, a checkpoint commitment is defined from the checkpoint bundle inputs rather than from a standalone Merkle-root pair.

$$Pk = (version, domain, Mk, k)$$

The checkpoint commitment HP is computed as:

In the reference construction, $HP = H_CHCK(Enc(checkpoint_header) \parallel block_merkle_root \parallel previous_checkpoint_commitment)$.

$$HP = H(Enc(Pk))$$

Checkpoints serve as verifiable reference points for recovery, audit, and long-term storage.

8.8 Seals

A seal is a cryptographic artifact that attests to a specific ledger state or checkpoint.

In the reference construction, the checkpoint commitment itself is the sealed state identifier. Optional deployment profiles may define additional seal structures, but checkpoint sealing signs the checkpoint commitment.

$$Z = (version, domain, T)$$

The seal commitment HZ is computed as:

$$HZ = H(Enc(Z))$$

If authentication is required, a digital signature σ is computed over either HZ or $Enc(Z)$, as defined by the sealing profile.

8.9 Anchors

An anchor binds a ledger state to externally defined reference data.

Let AR denote the canonical anchor reference containing external context data. An anchor commitment binds AR to a checkpoint commitment.

$$Ak = (version, domain, Mk, X)$$

The anchor commitment HA is computed as:

$$HA = H(Enc(Ak))$$

Optional authentication data MAY be applied according to the anchoring profile.

8.10 Verification Predicates

Verification of a ledger state $Sk = (Mk, k)$ succeeds if and only if all of the following conditions hold:

1. All records Ri are canonically serialized.
2. All record commitments satisfy:

$$Ci = H(Enc(Ri))$$

3. The recomputed Merkle root Mk' satisfies:

$$Mk' = Mk$$

4. The recomputed record count equals k .

Verification of checkpoints, seals, and anchors additionally requires validation of the corresponding commitment hash and any associated authentication material.

8.11 Security Reduction

The integrity and immutability guarantees of MCEL reduce to the collision resistance and second-preimage resistance of the hash function H , together with strict enforcement of canonical serialization and ordered commitment.

Any successful attack that modifies, removes, reorders, or substitutes records without detection implies a violation of these assumptions.

9. MCEL API

This section defines the application programming interface (API) for the Merkle Chain Evidence Ledger (MCEL). The API described here specifies the externally observable behavior of conforming implementations rather than prescribing a particular programming language, naming convention, or calling syntax. The intent is to define a stable interface model that exposes the MCEL ledger, its state transitions, and its derived artifacts in a manner consistent with the architectural and mathematical definitions given in Sections 5 through 8.

The API is normative in terms of semantics and behavior. An implementation is considered conforming if it provides equivalent functionality and produces artifacts that satisfy the verification predicates defined in Section 8.

9.1 API Model and Design Rationale

The MCEL API is structured around explicit state objects and deterministic operations. Ledger state is treated as a first-class object, and all operations that modify state do so through well-defined transitions. Derived artifacts, such as checkpoints, seals, and anchors, are exposed as immutable objects that can be independently verified.

The API deliberately avoids implicit global state. All operations are scoped to an explicit ledger context, which binds together the domain identifier, Merkle construction state, and any associated storage configuration. This design enables multiple independent ledgers to coexist within a single process or system without ambiguity.

9.2 Ledger Context and Core Types

A conforming implementation SHALL expose an abstract ledger context object that represents an initialized MCEL ledger bound to a specific domain. This context encapsulates the current ledger state, including the Merkle root and record count, and provides the entry point for all state-mutating operations.

In addition to the ledger context, the API SHALL expose abstract representations of records, cryptographic hashes, checkpoints, seals, and anchors. The concrete representation of these types is implementation-defined; however, their semantic meaning and behavior are fixed by this specification.

These core types correspond directly to the mathematical objects defined in Section 8 and SHALL preserve their semantics under serialization, comparison, and verification.

9.3 Ledger Initialization and Lifecycle

The API SHALL provide an initialization operation that creates or opens a ledger instance. This operation binds the ledger to a domain identifier and establishes the initial Merkle state. Where

persistent storage is enabled, initialization also establishes the storage context and validates any existing on-disk state.

The reference API exposes `mcel_ledger_initialize`, which validates storage callbacks, binds the namespace identifier, stores the public verification key, and verifies any existing head checkpoint bundle. If a non-empty head exists and verification fails, initialization fails.

Initialization behavior **MUST** be deterministic and policy-driven. If a ledger for the specified domain already exists, the implementation **SHALL** either open and verify the existing state or reject initialization, depending on the configured startup mode. In all cases, initialization **SHALL** fail if the existing state cannot be verified.

Implementations **MAY** provide an explicit shutdown or finalization operation to release resources associated with a ledger context. Such operations **SHALL NOT** modify ledger state or invalidate previously generated artifacts.

9.4 Record Construction and Append Semantics

The API **SHALL** provide a mechanism to construct records by supplying record metadata and an opaque payload. Record construction is a preparatory operation and **SHALL NOT** modify ledger state.

The append operation stores a record and returns its record commitment. The reference API exposes `mcel_ledger_append_record` for record append, `mcel_ledger_seal_block` for block sealing, and `mcel_ledger_seal_checkpoint` for checkpoint sealing. The ledger head changes only after checkpoint sealing succeeds.

If persistent storage is enabled, the append operation **SHALL** ensure that record data and updated ledger metadata are written durably in a manner that preserves atomicity. If any step of the append operation fails, the ledger state **SHALL** remain unchanged.

On successful completion, the append operation **SHALL** return the updated ledger state or provide access to the new Merkle root and record count through the ledger context.

9.5 Merkle State Access and Verification Support

The API **SHALL** expose accessors that allow retrieval of the current Merkle root and record count. These accessors provide the primary interface for applications and external systems to observe ledger state.

To support verification and audit, implementations **MAY** provide functions to construct Merkle inclusion proofs for individual records and to verify such proofs against a specified Merkle root. Any such functionality **SHALL** conform to the Merkle construction rules defined in Sections 6 and 8.

9.6 Record Query and Search Semantics

This section defines optional query capabilities that an implementation MAY provide for local record discovery and inspection. Query functionality is a read only facility. It SHALL NOT modify ledger state, record serialization, Merkle commitments, checkpoint material, or verification behavior. Query support is an implementation convenience layer and is not part of the cryptographic core of MCEL.

9.6.1 Scope and Model

An implementation that supports search SHALL expose a filter-based query model. A query evaluates a predicate over the ordered set of records in a ledger and returns either:

- a) A set of record references that satisfy the predicate, or
- b) A count of records that satisfy the predicate.

Record references SHALL correspond to stable append order positions within the ledger. Returned references SHALL reflect the canonical append only ordering of records.

The evaluation of a query SHALL be deterministic with respect to ledger state and filter parameters.

9.6.2 Filter Structure

A conforming query filter MAY include constraints over record header metadata and, optionally, over a bounded prefix of the record payload.

Header Constraints

An implementation MAY support equality or range constraints over header fields that are stable and committed within the record structure. At minimum, the following classes of constraint MAY be supported:

- a) Record type equality matching.
- b) Record flags matching, either full equality or masked comparison.
- c) Key identifier equality matching, where records are associated with a stable key identifier at append time.
- d) Timestamp range matching, expressed as inclusive lower and upper bounds.

If multiple header constraints are provided, the record SHALL satisfy all specified constraints to be considered a match.

9.6.3 Payload Prefix Matching

An implementation MAY support matching over a fixed length prefix of the record payload. Because payload content is application defined and opaque at the protocol level, payload

matching is explicitly non normative for interoperability. It SHALL be treated as a local selection heuristic only.

If supported, payload prefix matching SHALL:

- a) Compare at most a fixed maximum number of bytes.
- b) Operate on the raw payload as serialized in the record.
- c) Require exact byte equality over the specified prefix length.

An implementation SHALL enforce an upper bound on the prefix length to prevent excessive memory comparison cost.

9.6.4 Pagination Semantics

Query execution MAY support pagination controls consisting of:

- a) offset, the number of matching records to skip before returning results.
- b) limit, the maximum number of matching records returned.

If provided, limit SHALL be bounded by a fixed implementation defined maximum. This bound SHALL exist to prevent unbounded allocation and denial of service conditions.

Pagination SHALL be applied after full predicate evaluation. That is, the logical result set consists of all records satisfying the filter. Offset and limit SHALL select a contiguous window within that ordered result set.

9.6.5 Query Operations

A conforming implementation that provides search support SHALL implement at least one of the following operations.

Query Execution

Query execution SHALL:

- a) Evaluate the filter predicate over the ledger.
- b) Apply pagination controls if present.
- c) Return an ordered collection of record references.

Returned references SHALL be in ascending append order. The operation SHALL allocate and return ownership of the result collection to the caller. If allocation fails or input parameters are invalid, the operation SHALL fail cleanly and SHALL NOT leave partially initialized result structures observable to the caller.

Query Count

Query count SHALL evaluate the same predicate semantics as query execution but SHALL return only the total number of matching records. Pagination parameters, if supplied, SHALL be ignored for count operations.

9.6.6 Evaluation Strategy

The correctness of query semantics is defined solely by the filter predicate over the ledger. Implementations MAY employ different evaluation strategies for performance optimization, including but not limited to:

- a) Full ledger scan, evaluating the predicate sequentially over all records.
- b) Auxiliary index assisted evaluation, where a precomputed mapping reduces the candidate set before predicate evaluation.

If an auxiliary index is used, it SHALL be treated as an optimization only. The presence or absence of an index SHALL NOT affect correctness. If an index is incomplete, unavailable, or inconsistent, the implementation SHALL fall back to a correct evaluation strategy rather than returning incorrect results.

9.6.7 Auxiliary Indexing (Optional)

An implementation MAY maintain one or more in memory auxiliary indexes to accelerate specific classes of query, such as key identifier equality matching.

An auxiliary index:

- a) SHALL map stable header attributes to one or more record positions.
- b) SHALL not modify ledger data structures.
- c) SHALL be re-constructible from ledger state.

Index maintenance is implementation defined. If records are appended after index construction, the index SHOULD be updated in a manner that preserves consistency with the ledger's append order.

Index structures are not committed within the Merkle tree and are not subject to inclusion proof verification. They are purely local performance artifacts.

9.6.8 Error and Safety Semantics

Query operations SHALL:

- a) Validate all input parameters, including bounds on prefix length, offset, and limit.
- b) Enforce implementation defined maximums for allocation and result size.
- c) Fail deterministically and explicitly on invalid input or resource exhaustion.

Query failure SHALL NOT alter ledger state, record ordering, or cryptographic commitments. Query functionality SHALL be side effect free with respect to the ledger.

9.6.9 Security Considerations

Because payload matching operates on application defined data, it SHALL NOT be interpreted as a semantic validation mechanism. Authenticity and integrity of records remain governed exclusively by record signatures and Merkle based inclusion verification.

Implementations SHALL ensure that query support does not introduce unbounded memory growth, quadratic comparison behavior, or state mutation paths that could undermine the append only guarantees of MCEL.

9.7 Checkpoint Creation and Use

The API SHALL provide a mechanism to create checkpoints that capture a specific ledger state without modifying the ledger itself. Checkpoint creation involves constructing the checkpoint structure, canonically serializing it, and computing its cryptographic commitment.

The checkpoint API computes `H_CHK` over the encoded checkpoint header, block Merkle root, and previous checkpoint commitment, signs the resulting 32-byte commitment, stores the checkpoint bundle, and updates the head only after successful persistence.

Checkpoints are immutable artifacts. Once created, they SHALL NOT change and SHALL remain valid as long as the underlying ledger state remains valid. The API SHALL also provide a mechanism to verify a checkpoint artifact by recomputing its commitment and validating the referenced ledger state.

Checkpoint creation and verification are intended to support efficient recovery, long-term storage, and audit workflows, and do not alter ledger semantics.

9.8 Sealing Operations

The reference API provides checkpoint sealing as the concrete sealing mechanism. The signed checkpoint commitment is the seal over the checkpoint state. Generic seal structures may be defined by profiles, but the mandatory reference artifact is the signed checkpoint bundle.

Seals are immutable and verifiable artifacts. The API SHALL provide a mechanism to verify seals by recomputing the seal commitment and validating any associated authentication material. Sealing operations SHALL NOT modify ledger state.

9.9 Anchoring Operations

The API SHALL provide a mechanism to create anchors that bind a ledger state to externally defined reference data. Anchor creation involves constructing the anchor structure, canonically serializing it, and computing the anchor commitment. Optional authentication material MAY be applied according to the anchoring profile.

Anchor verification SHALL confirm both the correctness of the anchor commitment and the validity of the referenced ledger state. Anchors provide external verifiability but do not affect the internal operation of the ledger.

9.10 Verification and Audit Interfaces

The API SHALL provide verification functions that enable independent validation of ledger integrity. At a minimum, these functions SHALL support full ledger verification from records and checkpoint-based verification.

In addition, the API SHALL support deterministic traversal of ledger records and derived artifacts for audit purposes. The order of traversal SHALL be stable and reflect the append-only semantics of the ledger.

9.11 Error Reporting and Failure Semantics

All API operations SHALL report errors explicitly. Error conditions SHALL distinguish between invalid input, state violations, verification failures, and storage or I/O errors.

Under no circumstances SHALL an API operation leave the ledger in a partially updated or inconsistent state. State-modifying operations SHALL either complete successfully in their entirety or fail without observable side effects.

9.12 Conformance Requirements

An implementation is considered conformant with this specification if it exposes API behavior equivalent to that described in this section and if all generated artifacts satisfy the verification predicates defined in Section 8.

10. Storage and Persistence Model

This section specifies the storage and persistence requirements for conforming Merkle Chain Evidence Ledger (MCEL) implementations. The purpose of this section is to define the durability, consistency, and recovery properties required to preserve ledger integrity across restarts, failures, and untrusted storage environments. While the physical storage medium and file layout are implementation-defined, the behavioral guarantees described in this section are normative.

MCEL treats storage as a reliability mechanism rather than a trust anchor. All integrity guarantees are enforced cryptographically, and persisted data **MUST** remain verifiable regardless of the trustworthiness of the underlying storage.

10.1 Persistence Objectives

A conforming implementation **SHALL** ensure that persisted ledger artifacts satisfy the following objectives:

- Durability of appended records and derived commitments
- Deterministic recovery of ledger state following interruption
- Resistance to partial writes and torn updates
- Verifiable detection of storage corruption or tampering

Persistence mechanisms **SHALL NOT** weaken the append-only semantics or integrity guarantees defined elsewhere in this specification.

10.2 Persisted Artifacts

An MCEL implementation that supports persistence **SHALL** store, at a minimum, the following artifacts:

The reference storage model uses four logical locations: `mcel/records` for serialized records, `mcel/blocks` for sealed blocks, `mcel/checkpoints` for the append-only checkpoint log, and `mcel/head` for the current head checkpoint bundle.

- Canonical serialized records in append order
- Ledger metadata sufficient to reconstruct ledger state
- Checkpoints, if checkpointing is supported
- Seals and anchors, if generated

Each persisted artifact **SHALL** be stored in a form that allows independent verification using the procedures defined in Section 8.

10.3 Storage Abstraction

This specification does not mandate a specific storage medium. Implementations MAY use file systems, block storage, databases, object stores, or other persistence mechanisms.

Regardless of the storage medium, the implementation SHALL present a logical append-only abstraction to the ledger core. Storage operations that would allow modification or deletion of previously persisted records SHALL NOT be exposed through the MCEL API.

10.4 Atomicity and Consistency Guarantees

State-modifying operations, in particular record append, MUST be performed atomically with respect to persistence.

State-modifying persistence SHALL preserve atomicity at the record, block, and checkpoint levels. A checkpoint head SHALL NOT be updated unless the checkpoint bundle has been created, signed, appended to the checkpoint log, and written to the head location successfully.

- The record data, updated Merkle state, and updated metadata are all durably persisted, or
- None of these artifacts are persisted and the ledger state remains unchanged

Intermediate or partially persisted states SHALL be detectable and SHALL NOT be accepted as valid ledger state.

Implementations MAY achieve atomicity through transactional storage, write-ahead logging, copy-on-write techniques, or other mechanisms appropriate to the storage medium.

10.5 Checkpoint Persistence

Checkpoints, when supported, SHALL be persisted as immutable artifacts. A persisted checkpoint SHALL include sufficient information to reconstruct the corresponding ledger state, including the Merkle root and record count.

Checkpoint persistence SHALL NOT modify ledger state. Checkpoints MAY be persisted independently of record append operations and MAY be retained indefinitely.

Implementations SHOULD ensure that at least one valid checkpoint remains recoverable at all times, even in the presence of failures during subsequent operations.

10.6 Recovery Procedure

A conforming implementation SHALL support deterministic recovery of ledger state following restart or failure.

The recovery procedure SHALL proceed as follows:

1. Identify the most recent valid checkpoint, if checkpoints are used
2. Verify the checkpoint commitment and associated metadata
3. Replay persisted records in append order from the checkpoint state
4. Recompute record commitments and Merkle state
5. Reconstruct the current ledger state and validate persisted metadata

If no checkpoints are present, recovery SHALL proceed from the initial empty ledger state.

Any persisted artifact that fails verification during recovery SHALL cause recovery to fail or the artifact to be excluded, according to implementation policy. Under no circumstances SHALL unverifiable data be incorporated into recovered ledger state.

10.7 Handling of Storage Corruption

Because storage is not assumed to be trustworthy, implementations SHALL detect storage corruption through cryptographic verification.

If corruption is detected in persisted records, checkpoints, or metadata, the implementation SHALL:

- Reject the corrupted artifact
- Prevent further state modification until recovery or remediation
- Preserve sufficient diagnostic information to support audit or investigation

The detection of corruption SHALL NOT result in silent truncation or repair of ledger data.

10.8 Truncation and Compaction

MCEL does not permit truncation of the logical ledger. Records, once appended, are part of the ledger history.

Implementations MAY support physical compaction or archival techniques, provided that such techniques preserve the ability to verify ledger state and history. Any compaction scheme SHALL preserve all information required to validate checkpoints, seals, and anchors.

10.9 Untrusted and Distributed Storage

MCEL may be deployed on untrusted or distributed storage systems. In such environments, storage nodes MAY be assumed to be malicious or unreliable.

In these cases, MCEL relies exclusively on cryptographic verification to detect tampering. Storage availability and redundancy are operational concerns and are outside the scope of this specification.

10.10 Conformance

An implementation conforms to this section if it provides persistence behavior equivalent to that described above and if recovered ledger state satisfies the verification predicates defined in Section 8.

11. Security Analysis

This section analyzes the security properties of the Merkle Chain Evidence Ledger (MCEL) with respect to integrity, immutability, ordering, and evidentiary robustness. The analysis is structured around the architectural model (Section 5), the protocol objects (Section 6), the operational flows (Section 7), and the mathematical definitions and verification predicates (Section 8). Unless stated otherwise, the security claims in this section assume correct implementation of canonical serialization, correct implementation of the Merkle construction, and the continued collision and second-preimage resistance of the hash function used by MCEL.

MCEL is designed as an append-only, verifiable state machine, not as a distributed consensus protocol. The security properties of MCEL are therefore expressed in terms of tamper evidence, auditability, and verifiability under adversarial storage and transport, rather than in terms of distributed agreement.

11.1 Security Objectives

MCEL is intended to satisfy the following objectives:

- Provide cryptographically verifiable integrity for records and for the complete ledger state.
- Enforce record ordering and append-only semantics in a manner that is independently verifiable.
- Enable third parties to validate ledger state without trusting the storage medium, transport channel, or ledger operator.
- Support durable evidentiary artifacts (checkpoints, seals, anchors) that remain verifiable over time.
- Detect corruption, tampering, removal, substitution, and reordering of records and derived artifacts.

Confidentiality is not a native objective of MCEL. If confidentiality is required, it SHALL be provided by higher-level systems and SHALL NOT modify canonical serialization or commitment inputs in a way that breaks verifiability.

11.2 Threat Model and Assumptions

MCEL assumes an adversary may have one or more of the following capabilities:

- Read and copy ledger artifacts, including records, checkpoints, seals, and anchors.
- Modify, delete, reorder, truncate, or substitute persisted artifacts in storage.
- Replay previously observed artifacts to a verifier.

- Provide a verifier with a partial ledger history, a fabricated history, or inconsistent histories presented to different verifiers.
- Influence transport, including dropping, delaying, or reordering transmitted ledger artifacts.

MCEL does not assume that the adversary can break the underlying hash function. It does not assume secrecy of record contents or commitments. Where digital signatures are used for seals or anchors, MCEL assumes the authenticity and integrity of the signing key is maintained by the operational environment.

MCEL does not itself provide protection against a fully malicious writer who intentionally generates false or misleading records. MCEL ensures such records are immutable once appended, not that their contents are truthful.

11.3 Record Integrity and Tamper Evidence

Each record commitment binds the canonical serialized record to a cryptographic hash. Under collision resistance and second-preimage resistance, an adversary cannot feasibly construct a distinct record that yields the same commitment. Therefore, any modification to record header fields or payload bytes is detectable by recomputing the record commitment during verification.

Because the record commitment is incorporated into the Merkle construction, record integrity extends transitively to ledger state. A record cannot be modified without changing the Merkle root of any ledger state that includes it.

If canonical serialization is not implemented consistently, integrity guarantees degrade sharply. For this reason, canonical serialization is a security-critical requirement, and implementations SHALL reject records that cannot be serialized deterministically.

11.4 Ordering and Append-Only Properties

MCEL enforces ordering by committing to an ordered sequence of record commitments in the Merkle construction. The Merkle root commits not only to the set of records but to their order. Under the assumptions above, an adversary cannot reorder records without producing a different Merkle root.

Append-only semantics are enforced operationally by requiring that new state transitions only occur through the append operation, which produces a new Merkle root and increments the record count. A verifier that checks both the Merkle root and the record count can detect truncation, insertion, and reordering.

If an adversary attempts to present a truncated ledger, the record count binding (and optionally checkpoint binding) enables detection provided the verifier has an expected state reference (checkpoint, seal, or anchor) for comparison.

11.5 Ledger State Integrity and Immutability

The durable ledger state identifier is the checkpoint commitment together with its checkpoint header. The block Merkle root and record count identify the records covered by a block, while the checkpoint commitment binds that block root into the previous checkpoint chain.

Immutability in MCEL is therefore a property of verifiability rather than physical storage. MCEL does not prevent an adversary from changing stored data, but it ensures that any such change is detectable by verification.

The security of this property reduces to the collision and second-preimage resistance of the hash function and to the determinism of Merkle construction rules.

11.6 Checkpoints and Recovery Security

Checkpoints are derived artifacts that bind to a specific ledger state. They provide two security-relevant functions:

1. **Recovery assurance:** A checkpoint enables reconstruction of ledger state following interruption by replaying records from a known valid state.
2. **Audit acceleration:** A checkpoint provides a verifiable intermediate reference point, reducing the need to replay the full history.

A checkpoint commitment is itself a hash over the canonical serialization of checkpoint contents. If a checkpoint is altered, the checkpoint commitment changes and verification fails.

However, a checkpoint is not a trust anchor by itself. It is only meaningful when it can be verified against ledger data or compared against an externally validated reference (seal or anchor). If a verifier receives only a checkpoint without access to underlying records, the verifier can validate checkpoint integrity but cannot validate that the ledger history leading to that checkpoint is complete unless inclusion proofs or additional attestations are provided.

11.7 Seals and Attestations

Seals provide an attestation over a ledger state or checkpoint. The seal commitment binds together the referenced state identifier and associated metadata. Seals improve evidentiary robustness by creating durable artifacts that can be archived, compared, and audited independently.

If seals are unsigned, they provide integrity but not authentication. In this case, they are valuable as stable identifiers but do not establish who produced the seal.

If seals are signed, they provide authenticity and non-repudiation relative to the signing key. In this case, an adversary without the signing key cannot forge valid seals for arbitrary ledger states.

The security of signed seals depends on correct signature verification and on operational key protection. MCEL treats signature schemes as an optional authentication layer. If deployed, signature policy and key management SHALL be clearly specified as an implementation profile.

11.8 Anchoring and External Verifiability

Anchors bind a ledger state to an external context. Anchoring is intended to provide evidence that a particular Merkle root existed at or before a reference event.

Anchors improve security against equivocation by a ledger operator when verifiers can compare anchored states across time or across parties. If different parties are presented with different ledger histories, divergence is detectable when anchored commitments are compared.

Anchors are not equivalent to consensus. Anchoring does not prevent a malicious writer from maintaining multiple divergent histories, but it increases the cost of sustaining equivocation in environments where anchors are published to widely visible channels or independently retained by multiple verifiers.

Where anchors are signed or published into external immutable media, the anchor can serve as a strong evidentiary artifact supporting time-bounded existence claims.

11.9 Forking and Equivocation Resistance

Because MCEL is not a consensus protocol, a malicious writer can create two different ledgers (or two branches of the same logical domain) and present different histories to different parties. This is an inherent limitation of single-writer append-only systems.

MCEL provides detection mechanisms rather than prevention mechanisms:

- If two parties compare Merkle roots for the same claimed state index and they differ, equivocation is detected.
- If anchors or seals are shared across parties, divergence becomes externally visible and therefore detectable.
- If a party maintains periodic local checkpoints and compares them against presented states, divergence is detected.

Operationally, equivocation resistance requires policies around publication, distribution, and retention of anchored states, which are outside the scope of MCEL core construction but are supported by MCEL artifacts.

11.10 Replay, Substitution, and Context Confusion

MCEL is designed to resist replay and substitution attacks by using cSHAKE domain separation, versioned structures, key identifiers, record sequence numbers, checkpoint-chain linkage, and

externally retained reference states. The namespace identifier is an operational ledger identifier; cryptographic object-type separation is performed by the cSHAKE customization strings.

A substitution attack attempts to replace one record or artifact with another. Such substitution is detectable if:

- The substituted object yields a different commitment hash, or
- The substituted object is in a different domain, and domain binding is enforced, or
- The substituted object belongs to a different protocol version and version binding is enforced.

Replay attacks against verifiers are mitigated by comparing presented ledger state identifiers against expected references (for example, a known seal or anchor). If a verifier does not maintain an expected reference, replay of older valid states cannot be detected, because the older state remains cryptographically valid. This is not a weakness of MCEL, it is a consequence of immutable history. Detection of staleness requires an external freshness policy.

11.11 Truncation and Omission Attacks

An omission attack presents a verifier with a partial ledger, excluding some records, while claiming completeness.

If the verifier has a reference state, such as a checkpoint commitment, signed checkpoint bundle, or retained anchor commitment, truncation is detected by mismatch of checkpoint commitment, previous-commitment linkage, block root, or record count.

If the verifier lacks a reference state and accepts the presented root as authoritative, truncation cannot be detected in principle, because the presented ledger is internally consistent. Therefore, security against omission requires that verifiers obtain state references through seals, anchors, or other externally validated checkpoints.

This is a fundamental aspect of append-only ledger verification: integrity is internal, completeness is external unless a trusted reference is available.

11.12 Hash Function and Long-Term Integrity

MCEL's security reduces primarily to the strength of the underlying hash function. If collision resistance is weakened, an adversary may construct alternative records or alternative Merkle trees that result in the same root, undermining integrity.

If second-preimage resistance is weakened, an adversary may be able to replace a record with another record yielding the same commitment, undermining immutability.

For long-term evidentiary deployments, MCEL SHOULD use hash functions with conservative security margins and SHOULD support algorithm agility via versioned data structures. Implementations SHOULD define migration strategies for updating hash functions, including re-anchoring and sealed checkpoints at migration boundaries.

11.13 Implementation Risks and Requirements

MCEL security depends on correct implementation of:

- Canonical serialization
- Deterministic Merkle construction rules
- Strict domain binding
- Correct persistence atomicity and recovery verification
- Correct verification of optional signature schemes

Implementation errors in serialization and hashing are the most serious risks because they can produce inconsistent commitments across platforms or enable ambiguity in verification.

Implementations SHALL include self-tests and cross-platform test vectors to confirm identical commitment outcomes for identical inputs.

Persistence errors can create inconsistent recovered states. Recovery MUST verify every reconstructed commitment and MUST reject unverifiable artifacts.

11.14 Residual Risks and Operational Controls

MCEL does not address the truthfulness of record contents, only their immutability after recording. Systems relying on MCEL for evidentiary purposes SHALL define operational controls governing:

- Who is authorized to append records
- How records are sourced and validated prior to append
- How anchors and seals are generated, published, and retained
- How verifiers obtain trusted state references
- How keys are protected when signatures are used

MCEL provides the cryptographic mechanism for tamper evidence and auditability. The operational environment defines how that mechanism is used to achieve legal, regulatory, or institutional assurance.

12. Cryptanalysis Considerations

This section examines MCEL from a cryptanalytic perspective. The goal is not to prove security in a formal model, but to clearly delineate the attack surfaces, reduction boundaries, and failure modes relevant to the construction. The analysis focuses on hash-based commitments, Merkle aggregation, ordered state transitions, and derived artifacts such as checkpoints, seals, and anchors.

Unless otherwise stated, the analysis assumes correct implementation of canonical serialization, deterministic Merkle construction, and correct domain separation, as defined in earlier sections.

12.1 Scope of Cryptanalysis

MCEL is not a cipher, key exchange protocol, or authentication protocol. Consequently, cryptanalysis of MCEL does not concern secrecy, indistinguishability, or key recovery. Instead, the relevant cryptanalytic questions are:

- Whether an adversary can modify ledger contents without detection
- Whether an adversary can construct alternative histories with identical commitments
- Whether ordering, inclusion, or omission can be falsified cryptographically
- Whether derived artifacts can be forged or confused across contexts

All cryptanalytic considerations reduce to properties of the underlying hash function and to structural properties of the Merkle construction.

12.2 Hash Function Dependency and Reduction Boundary

MCEL relies on cSHAKE-256 for integrity commitments. Although the primitive is common, the construction uses distinct cSHAKE customization strings for payload, record, node, block, checkpoint, and anchor commitments. A successful cross-object substitution must therefore defeat the relevant cSHAKE domain separation or find a collision in the applicable domain.

- Collision resistance
- Second-preimage resistance
- Preimage resistance (to a lesser extent)

If collision resistance fails, an adversary may be able to construct two distinct inputs that yield the same hash value. In the context of MCEL, this could enable the construction of two distinct records or two distinct Merkle subtrees that result in the same commitment. Such a failure would undermine ledger integrity.

If second-preimage resistance fails, an adversary may be able to replace an existing record with a different record that yields the same record commitment. This would directly violate immutability guarantees.

MCEL does not rely on pseudo-randomness, secrecy, or key material in its core construction. Therefore, the security reduction boundary is narrow and well defined.

12.3 Record-Level Attacks

At the record level, the primary cryptanalytic objective for an adversary is to substitute one record for another without changing the record commitment.

Given a record with encoded header RH , payload commitment PC , and record commitment $RC = H_RCRD(Enc(RH) \parallel PC)$, an adversary attempting record substitution must find a different header or payload commitment that yields the same RC , or a different payload that yields the same PC under the appropriate payload domain.

$$C_i = H(Enc(R'_i))$$

This is a second-preimage attack against H under structured input. MCEL provides no additional entropy or salt at the record level beyond what is present in the record itself. Consequently, the feasibility of such an attack depends entirely on the second-preimage resistance of H .

Domain identifiers, version fields, and flags are included in canonical serialization specifically to increase structural diversity and to reduce the risk of cross-context substitution.

12.4 Merkle Tree Collision and Structure Attacks

The Merkle construction aggregates record commitments into a single root. The cryptanalytic goal at this level is to produce two distinct ordered commitment sequences that yield the same Merkle root.

This requires either:

- A collision in H at an internal node, or
- A structural ambiguity in the Merkle construction rules

The first case reduces directly to collision resistance of H . The second case arises if Merkle construction rules are underspecified or inconsistently implemented. For example, ambiguity in handling odd numbers of nodes can create divergent trees across implementations.

MCEL explicitly requires the duplicate-last rule for unpaired nodes. An implementation that promotes an unpaired node without hashing or uses any other odd-node rule will produce different roots and is non-conforming.

12.5 Ordering and Position-Based Attacks

Because MCEL commits to an ordered sequence of records, an adversary may attempt to reorder records while preserving the same Merkle root.

Such an attack would require constructing two different ordered sequences of commitments that produce the same Merkle root. This again reduces to a collision attack on H , because the Merkle construction is order-sensitive.

There is no known structural shortcut that allows reordering without inducing a hash collision, assuming deterministic construction and collision resistance.

12.6 Prefix, Extension, and Length-Extension Considerations

MCEL hash inputs are always derived from canonical serialization of structured data or concatenation of fixed-length hash outputs.

Because cSHAKE-256 is a sponge-based SHA-3 derived function and MCEL uses fixed-size child hashes or canonical encodings, the construction is not exposed to Merkle-Damgård length-extension behavior.

Provided that:

- $\text{Enc}(\cdot)$ produces unambiguous byte strings, and
- Hash concatenation is performed over fixed-length inputs,

MCEL is not vulnerable to classical length-extension attacks that affect some Merkle–Damgård hash constructions. The Merkle tree operates on hash outputs, not raw message blocks, and therefore does not expose internal chaining state.

12.7 Chosen-Input and Adaptive Attacks

An adversary may be able to choose record contents adaptively, observing resulting record commitments and Merkle roots.

MCEL does not attempt to hide structure or values. Observability of commitments does not weaken integrity guarantees, because the attacker’s goal is to cause an undetected modification, not to predict a hash output.

Adaptive chosen-input capability does not materially reduce security beyond the baseline assumptions about H . Any attack that succeeds under adaptive input also implies a fundamental weakness in H .

12.8 Checkpoint, Seal, and Anchor Forgery

Checkpoints, seals, and anchors are all hash-based commitments over structured data. Forgery of these artifacts without detection requires either:

- Producing a collision in H , or
- Exploiting ambiguity in canonical serialization or structure definition

Unsigned seals and anchors provide integrity but not authenticity. An adversary may create a new seal or anchor for a ledger state they control, but cannot create a seal or anchor that falsely attests to a different ledger state without breaking H .

When digital signatures are applied, forgery additionally requires breaking the underlying signature scheme or compromising key material. This is orthogonal to MCEL's hash-based security and depends on external cryptographic assumptions.

12.9 Equivocation and Multi-History Attacks

From a cryptanalytic standpoint, equivocation does not require breaking cryptography. A malicious writer can maintain multiple distinct ledgers and present them selectively.

MCEL does not attempt to cryptographically prevent equivocation. Instead, it ensures that equivocation is detectable when ledger states are compared. Cryptanalysis therefore treats equivocation as an operational threat, not a cryptographic one.

Anchors and seals increase the cost of equivocation by creating durable commitments that can be compared across time and parties.

12.10 Algorithm Agility and Hash Migration

Because MCEL's security reduces to the strength of H , long-term deployments must consider the possibility of future cryptanalytic advances.

MCEL supports algorithm agility through versioned structures. A migration strategy typically involves:

- Finalizing the ledger state under the old hash function
- Creating a checkpoint and seal
- Reinitializing a new ledger under a new hash function
- Anchoring the transition state

Cryptanalysis of MCEL does not assume perpetual hash security. Instead, it assumes that migration is possible and detectable.

12.11 Side-Channel and Implementation Considerations

MCEL's cryptographic operations are limited to hashing and optional signature verification. Side-channel attacks such as timing or cache leakage are generally not applicable to ledger integrity, because no secrets are involved in hash computation.

However, implementation-level errors can introduce vulnerabilities that are not cryptanalytic in nature, including:

- Non-canonical serialization
- Inconsistent Merkle construction
- Improper handling of persistence and recovery
- Silent acceptance of verification failures

Such errors can undermine security guarantees without breaking cryptography and must be addressed through testing, review, and validation.

12.12 Summary of Cryptanalytic Posture

From a cryptanalytic perspective, MCEL is a narrow construction with a well-defined security reduction. Its integrity guarantees reduce almost entirely to the properties of the underlying hash function and to the determinism of its structural rules.

There are no known shortcut attacks that bypass hash security assumptions. Any successful undetected modification, substitution, or reordering of records implies a fundamental weakness in the hash function or a violation of the specification.

This constrained reduction surface is a deliberate design choice and is one of the primary strengths of the MCEL construction.

13. Deployment Considerations

This section describes practical considerations for deploying the Merkle Chain Evidence Ledger (MCEL) in real-world systems. The intent is to explain how MCEL is expected to be operated, integrated, and governed so that the cryptographic guarantees defined in earlier sections translate into meaningful operational assurance. While this section is not concerned with cryptographic construction, improper deployment choices can materially weaken the evidentiary value of an otherwise correct implementation.

MCEL is designed to be deployment-agnostic. It can be used in small, embedded environments as well as in large institutional systems. In all cases, its security properties rely on clearly defined trust boundaries, disciplined operational procedures, and consistent verification practices.

13.1 Roles, Responsibilities, and Trust Boundaries

An MCEL deployment implicitly defines several roles, even if they are realized by the same organization or system. At minimum, there is an entity responsible for appending records, an entity responsible for storing ledger artifacts, and one or more entities that verify ledger state. These roles need not trust each other beyond what is enforced cryptographically.

MCEL assumes that verifiers do not trust storage by default. A verifier should treat all ledger artifacts as potentially adversarial until they have been verified against expected commitments. Where the same organization performs writing, storage, and verification, this separation of roles is conceptual rather than organizational, but the verification discipline remains important.

Clear definition of these roles at deployment time is critical, especially in regulated or evidentiary contexts. Ambiguity about who is authorized to append records or who is responsible for retaining anchors and checkpoints can undermine the system's intended guarantees.

13.2 Writer Models and Append Control

MCEL is designed for environments where append authority is controlled by policy rather than by distributed consensus. In the simplest case, a single writer is responsible for all append operations. In more complex environments, multiple writers may exist, but their operations must be serialized deterministically.

MCEL itself does not arbitrate concurrent appends. If multiple writers are permitted, deployments must provide an external mechanism to ensure that records are appended in a well-defined order. Failure to do so risks producing divergent ledger states that are individually valid but mutually inconsistent.

The choice between a single-writer and controlled multi-writer model should be made explicitly and documented as part of the deployment architecture.

13.3 Integration into Larger Systems

MCEL is not a complete application. It is intended to be embedded within larger systems that provide application semantics, access control, and business logic. In such integrations, MCEL should be treated as the integrity and ordering substrate, not as a general-purpose database.

Applications should assume that records, once appended, are immutable. If application-level state is expected to change over time, those changes should be represented as new records rather than as modifications to existing records. Care should be taken when embedding references to external objects or systems inside record payloads, as the meaning of such references may change over time even though the ledger record does not.

Long-lived deployments should consider how future readers will interpret historical records, especially when application semantics evolve.

13.4 Storage Environments and Availability

MCEL can be deployed on trusted storage, untrusted storage, or a mixture of both. The cryptographic guarantees are the same in all cases, but the operational risks differ.

On trusted storage, MCEL primarily protects against accidental corruption and internal error. On untrusted or third-party storage, MCEL additionally protects against deliberate tampering, deletion, and substitution. In such environments, verification should be treated as routine rather than exceptional.

Availability is not a cryptographic property of MCEL. Replication, backup, and redundancy strategies are operational decisions that strongly affect system reliability but are outside the scope of this specification. However, deployments that rely on MCEL for evidentiary purposes should ensure that ledger artifacts and reference anchors remain accessible for as long as they may be needed.

13.5 Checkpoints, Seals, and Anchors in Practice

The practical value of checkpoints, seals, and anchors depends on how they are generated, retained, and distributed. Creating these artifacts without a retention or publication strategy provides limited benefit.

Deployments should define how frequently checkpoints are created, under what conditions seals are applied, and where anchors are stored or published. In many environments, the strongest assurance is achieved when anchors are retained independently of the primary ledger storage and are available to multiple parties.

These artifacts are most valuable when they are treated as durable reference points rather than as transient implementation details.

13.6 Verification Discipline

MCEL's security guarantees are realized only when verification is performed. A deployment that appends records but rarely verifies ledger state is relying on cryptography without exercising it.

Verification may be performed at different times and at different depths depending on risk tolerance. Some environments may verify every append operation, while others may rely on periodic verification or verification triggered by audits or disputes. Regardless of frequency, verification should be treated as a normal operational activity, not as an exceptional event.

Failure to verify does not break MCEL, but it weakens its practical effectiveness as an integrity mechanism.

13.7 Time, External References, and Interpretation

MCEL itself has no notion of time. Any timestamps included in records, checkpoints, or anchors are application-defined and should not be treated as authoritative unless they are bound to a trusted external time source.

If time-based claims are important, deployments should use anchors to bind ledger state to external systems or services that provide independent time assertions. MCEL ensures that such bindings are immutable once created, but it does not validate the correctness of the external reference.

Care should be taken to clearly document how time and external references are interpreted by verifiers.

13.8 Versioning and Long-Term Operation

MCEL is designed to support long-term operation through explicit versioning of structures and algorithms. Deployments should plan for the possibility of future changes, including hash algorithm migration and format evolution.

When transitions occur, it is good practice to finalize existing ledger state with a checkpoint and, where appropriate, a seal or anchor. This creates a clear boundary between versions and preserves the interpretability of historical data.

Version transitions should be explicit and auditable, rather than implicit or ad hoc.

13.9 Evidentiary and Regulatory Contexts

When MCEL is used in legal, regulatory, or compliance settings, its cryptographic guarantees must be complemented by appropriate governance and operational controls. MCEL can demonstrate that records have not been altered and that their order is preserved, but it does not by itself establish authorship, intent, or truthfulness.

Deployments in such contexts should align MCEL usage with applicable standards of evidence, including documented procedures for append authorization, verification, and artifact retention. MCEL provides a strong technical foundation, but its evidentiary value ultimately depends on how it is operated.

13.10 Summary

MCEL is intentionally minimal and conservative at the cryptographic level. This design places greater importance on deployment discipline, documentation, and verification practice. When deployed with clear trust boundaries, controlled append behavior, consistent verification, and careful management of derived artifacts, MCEL provides strong and durable integrity guarantees. When these operational considerations are neglected, the cryptographic core remains sound, but the overall system assurance is diminished.

14. Compliance and Interoperability

This section defines what it means for an implementation to conform to the Merkle Chain Evidence Ledger (MCEL) specification and explains how independent implementations achieve interoperability. The emphasis is on externally observable behavior and verifiable artifacts rather than internal design choices. An implementation either produces artifacts that can be verified according to this specification or it does not. Internal architecture, programming language, storage layout, and API shape are explicitly out of scope for conformance.

Interoperability in MCEL is artifact-based. Two implementations interoperate successfully if artifacts produced by one can be verified by the other and if both derive identical ledger state identifiers from identical inputs.

14.1 Definition of Compliance

An implementation is considered compliant if it implements the mandatory components of this specification and if all ledger artifacts it produces satisfy the verification predicates defined in Section 8. Compliance is assessed by observation and verification, not by inspection of source code or internal structure.

A compliant implementation must correctly serialize record headers as 58-byte encodings, block headers and checkpoint headers as 62-byte encodings, use big-endian integer encoding, compute payload, record, node, block, checkpoint, and anchor commitments using the exact cSHAKE domains defined in this document, implement the duplicate-last Merkle rule, and verify signed checkpoint bundles before accepting checkpoint state.

Compliance is binary. An implementation that deviates in any way that produces unverifiable or ambiguous artifacts is non-compliant, even if it is internally consistent.

14.2 Canonical Serialization as the Primary Interoperability Constraint

Canonical serialization is the single most important interoperability requirement in MCEL. All higher-level guarantees, including record integrity, Merkle consistency, and artifact verification, depend on identical byte-level representations of logically identical objects.

Two independent implementations presented with the same logical record must produce identical serialized byte streams. Any divergence in field ordering, integer encoding, length representation, or handling of optional fields results in different record commitments and therefore different ledger states. Such divergence breaks interoperability completely, even if both implementations are internally correct.

For this reason, canonical serialization must be treated as a security-critical and interoperability-critical component. Implementations should not attempt to be permissive. Inputs that cannot be serialized deterministically must be rejected rather than normalized heuristically.

14.3 Deterministic Merkle Construction and State Identity

Interoperability further requires that Merkle construction rules be applied identically across implementations. The Merkle tree is not merely an optimization, it defines ledger state identity. Any variation in tree construction, including leaf definition, node combination order, or handling of odd node counts, results in different Merkle roots for the same record sequence.

An implementation that uses a different Merkle construction strategy, even if it appears functionally equivalent in isolation, will not interoperate with conforming implementations. Determinism is therefore mandatory, not advisory.

Block state is identified by the block Merkle root and record count. Durable ledger state is identified by the checkpoint commitment and checkpoint header. Interoperability requires two implementations given the same committed inputs to derive the same payload commitments, record commitments, block roots, block commitments, checkpoint commitments, and anchor commitments.

14.4 Artifact Compatibility Across Implementations

MCEL defines several artifact types, including records, checkpoints, seals, and anchors. Interoperability is achieved when these artifacts can be exchanged between systems and verified without ambiguity.

An implementation may use any internal representation it chooses, but any artifact exported for verification or exchange must conform to the canonical structure and serialization rules defined in this specification. Receiving implementations must verify artifacts cryptographically and must not rely on implicit trust, contextual assumptions, or out-of-band metadata.

Artifact compatibility is evaluated solely through verification outcomes. If an artifact verifies successfully under an independent implementation, it is interoperable. If it does not, it is not.

14.5 Error Handling and Interoperability Behavior

Predictable failure behavior is an important part of interoperability. When an implementation encounters invalid input, malformed artifacts, or verification failures, it must fail explicitly and unambiguously. Silent acceptance, partial acceptance, or best-effort interpretation undermines interoperability by masking incompatibilities.

While the specific error reporting mechanism is implementation-defined, failures should clearly distinguish between malformed input, verification failure, and internal error. This distinction is essential when diagnosing interoperability problems between independent systems.

Under no circumstances should a verification failure result in a partially accepted ledger state or an ambiguous outcome.

14.6 Versioning and Cross-Version Interaction

MCEL supports explicit versioning of structures and artifacts to allow controlled evolution of the specification. Version identifiers are part of canonical serialization and therefore part of the cryptographic commitment.

An implementation encountering an artifact with an unsupported version must reject it. Attempting to interpret or partially process unknown versions risks semantic confusion and verification errors. Backward compatibility, if provided, must be explicit and documented, and must not alter the interpretation of historical artifacts.

Interoperability across versions is therefore a policy decision layered on top of the core specification, not an implicit property of the protocol.

14.7 Interoperability Testing and Validation

Because MCEL interoperability is defined entirely in terms of verifiable artifacts, testing is straightforward in principle. One implementation produces artifacts, and another independently verifies them. If verification succeeds and derived ledger state identifiers match, interoperability is achieved.

In practice, providing shared test vectors greatly simplifies this process. Test vectors should include canonical serialized records, corresponding record commitments, Merkle roots for defined record sequences, and representative checkpoints, seals, and anchors. These vectors serve as a common reference point for independent implementations.

Cross-implementation testing is strongly encouraged for any deployment where interoperability is a requirement rather than a convenience.

14.8 Scope and Limits of Interoperability

MCEL defines interoperability at the level of cryptographic artifacts and verification behavior. It does not define interoperability for APIs, storage layouts, networking protocols, or application semantics.

Two implementations may expose entirely different APIs and use entirely different storage mechanisms while remaining fully interoperable, provided that the artifacts they produce and consume verify correctly and yield identical ledger state identifiers.

Conversely, two implementations with identical APIs but differing serialization or Merkle rules are not interoperable, regardless of surface similarity.

14.9 Summary

Compliance with this specification is determined by behavior and by verifiable output, not by implementation technique. Interoperability is achieved when independent implementations produce artifacts that can be mutually verified and when identical inputs lead to identical ledger state identifiers.

The strictness of MCEL's requirements, particularly around canonical serialization and deterministic Merkle construction, is intentional. These constraints are what allow MCEL to function as a reliable, system-independent integrity and audit substrate across organizations, platforms, and time.

15. Conclusion

The Merkle Chain Evidence Ledger (MCEL) defines a minimal, deterministic, and cryptographically grounded ledger construction for producing durable, verifiable evidence of ordered events. Rather than attempting to solve consensus, identity, or policy enforcement, MCEL deliberately focuses on a narrower and more defensible problem space: providing strong, portable guarantees of integrity, ordering, and auditability over append-only data.

At its core, MCEL is a cSHAKE-256 domain-separated evidence state machine. Records are committed through payload and record commitments, aggregated into deterministic Merkle blocks, and bound into a signed checkpoint commitment chain. The protocol's guarantees derive from collision and second-preimage resistance, strict canonical serialization, deterministic Merkle construction, and correct checkpoint verification.

The design emphasizes clarity of reduction and verifiability over architectural complexity. By avoiding implicit global state, hidden side effects, or non-deterministic behaviors, MCEL enables independent implementations to interoperate through artifact exchange and verification alone. Checkpoints, seals, and anchors extend this core model by providing durable reference points, attestations, and external bindings, without altering the underlying ledger semantics. These mechanisms allow MCEL to support long-lived deployments, recovery, and third-party audit while preserving a clean separation between cryptographic guarantees and operational policy.

A key strength of MCEL is that its security properties are explicit and inspectable. Integrity, ordering, and immutability are not emergent behaviors, they are directly encoded into the construction and can be verified independently by any party with access to the relevant artifacts. Conversely, properties that MCEL does not provide, such as consensus, confidentiality, or truthfulness of record contents, are intentionally left to higher-level systems. This clear boundary makes MCEL easier to reason about, easier to implement correctly, and easier to deploy in environments with well-defined trust models.

In real-world applications, MCEL is well suited to domains where evidentiary integrity and auditability are primary requirements. These include compliance and regulatory logging, software supply chain provenance, evidence and chain-of-custody systems, institutional attestations, trade and finance documentation, and long-term record retention where independent verification is required. MCEL is particularly effective in settings where storage or operators cannot be fully trusted, but where verifiers require strong, portable assurance that records have not been altered or reordered after the fact.

More broadly, MCEL can function as a foundational component within larger systems, acting as a cryptographic integrity substrate upon which identity, policy, access control, and application semantics are layered. Its simplicity and strict determinism make it suitable for both constrained environments and large-scale institutional deployments, while its artifact-based interoperability enables independent verification across organizational and technical boundaries.

In summary, MCEL represents a conservative and deliberate approach to ledger design. By narrowing its scope, formalizing its behavior, and grounding its guarantees in minimal cryptographic assumptions, it provides a robust and adaptable mechanism for evidentiary record keeping. When combined with appropriate operational discipline and governance, MCEL offers a practical and future-resistant foundation for systems that require trustworthy records and verifiable history.